

8. Bölüm

Nesne Yönelimli Programlama

8.1 Nesne Yönelimli Programlama Paradigması

Yapısal programlamada talimatlar (**statement**) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya **emreden programlama** (**imperative programming**) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.

Emreden programlamanın bir örneği olan yapısal programlamada talimatlar yine art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve 2.8 başlığı altında anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir. Bu dillerde **nesneler** (**object**) programın temel yapı taşlarıdır. Nesneler; **durum** (**state**) ve **davranışlara** (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine **ileti göndermesiyle** (**message-passing**) yapılır.

Nesne yönelimli programlamada **kontrol akışı** (**kontrol flow**), **nesnelerin birbirlerine yöntem çağrıları yaparak etkileşime girmesiyle** yönetilir.

1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan *Bjarne Stroustrup*, sonradan C++ olarak bilinecek olan sınıflarla C (**C with classes**) üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++ dili uzun yıllar donanımına yakınlığı nedeniyle ve yüksek performans isteyen uygulamalarda hala tercih edilmektedir. Ancak saf nesnelerden oluşan bir programlama dili değildir. Bir de kodlar her platform (işletim sistemi ve işlemci için) için yeniden derlenmelidir.

C# programlama dili saf nesne yönelimli programlama dili olup, Microsoft tarafından geliştirilmiştir. 2000 Yılında kapalı kaynak olarak geliştirilen bu dil, 2016 yılından sonra **açık kaynak** olarak yayınlanmıştır. C++ ve C dilinden daha fazla kütüphaneye sahiptir. .NET platformu için derlendiğinden geliştirilen uygulamalar, derlenmeden platformlar arası taşınarak çalıştırılabilmektedir.

C# dilinde, altyapının bize sunduğu kütüphanelerdeki hazır nesneleri kullanarak programları geliştiremeyiz. Mutlaka programcı tarafından ihtiyaçlara göre veri içeren ve davranış gösteren nesnelere ihtiyaç duyarız. İşte bu nesneler, **kullanıcı tanımlı sınıflar** (**user defined class**) ile tanımlanır.

Emreden paradigmanın aksine nesnelerin birbirlerine ileti göndermesi (yöntem çağrıları yaparak etkileşime girmesi) bakış açısıyla yapılan programlamaya **nesne yönelimli programlama** OOP (**object-oriented programming**) paradigması adı verilir.

8.2 Sınıf ve Nesne Nedir? Nasıl Tanımlanır?

Gerçek dünyamızda nesneleri sınıflamaya tabi tutarız. Örneğin bir birine benzeyen at, eşek ve zebra gibi hayvanları daha çocukluktan itibaren ayırt edebilir. İşte kafamızda oluşan bu mekanizmaya **sınıflayıcı** (**classifier**) adı verilir. Bu mekanizma ile sınıflanmış her bir türün gerçek dünyadaki her birine ise **örnek** (**instance**) adı verilir.

Nasıl ki dünyamızda evler bir mimari plan (**blueprint**) üzerinden inşa ediliyor, yazılımda kullanacağımız **nesneler** (**object**) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (**state**) ve nesnelerin göstereceği **davranışlarının** (**behavior**) tanımlandığı bu plana **sınıf** (**class**) adı verilir. Nesneler (**object**) yukarıda anlatılan gerçek dünya **örneklerine** (**instance**) karşılık gelir. Sınıflar ise **sınıflayıcıya** (**classifier**) karşılık gelir.

Bir mimari plandan birçok ev yapılabileceği gibi, **bir sınıftan da birçok nesne imal edilebilir.**

Gerçek dünyadaki nesneleri modelleyen **kullanıcı tanımlı sınıflar** (**user defined class**), aşağıdaki şekilde tanımlanır;

```
[erişim-değiştiricisi] class sınıf-kimliği
```

```
{
    [erişim-değiştiricisi] veri-tipi alanKimliği; // Alan değişkeni (field variable)
    [erişim-değiştiricisi] veri-tipi yöntemKimliği() // Davranış/Yöntem
    {
        // Yöntemler, yapısal programlamadaki fonksiyonlar gibi kodlanır.
    };
}
```

İmal edilecek nesnelerin durumları (**state**), sınıftaki alanlarla (**field**) tanımlanır. Nesnelerin göstereceği davranışlar da sınıf içinde fonksiyon gibi tanımlanır. **Aynı davranış, farklı yöntemlerle yerine getirilebileceğinden**, davranışları tanımlayan bu fonksiyonlara **yöntem (method)** adı verilir. İşin doğası gereği davranışlar durumlardan etkilenirler.

Özet olarak sınıflar, kodumuzda **imal edilecek nesnelere ilişkin ortak davranış ve durumları tasnif eden bir taslak bileşenidir.**

Kullanıcı Tanımlı sınıflar referans tip olduğundan bu sınıflardan, **new** işleci (**new operator**) ile nesne imal edilir. Bu husus 4.2.2 başlığında anlatılmıştı. Bir referans tipine ilişkin bellekte yer tahsisi, çalışma zamanında (**run-time**), öbek (**heap**) bellek üzerinde yapılır.

Şimdi ortak davranış ve özelliklerin tanımlandığı bir **Öğrenci** sınıfı ve bu sınıftan nesne imal eden ana programı kodlayalım;

```
1 //ANA PROGRAM:
2 Öğrenci ali = new Öğrenci();
3 ali.yaş = 40;
4 ali.YaşınıSöyle();
5 ali.CinsiyetiniSöyle();
6
7 //SINIF:
8 class Öğrenci // "Öğrenci" Kimlikli Sınıf
9 {
10     // Alanlar (Fields) isimleri için Deve Yazımı tercih edilir.
11     public uint yaş;
12     public char cinsiyet;
13
14     // Yöntem (Method) isimleri PascalCase olmalıdır.
15     public void YaşınıSöyle()
16     {
17         Console.WriteLine($"Yaşım:{yaş}");
18     }
19     public void CinsiyetiniSöyle()
20     {
21         switch (char.ToUpper(cinsiyet)) // Küçük/büyük harf karmaşasını tek satırda çözer.
22         {
23             case 'E':
24                 Console.WriteLine("Cinsiyetim: ERKEK");
25                 break;
26             case 'K':
27                 Console.WriteLine("Cinsiyetim: KADIN");
28                 break;
29             default:
30                 Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
31                 break;
32         }
33     }
34 }
```

Programı şöyle açıklayabiliriz;

- ◇ 3. Satırda Kullanıcı Tanımlı **Öğrenci** sınıfından **ali** kimlikli bir **nesne imal edilmiştir (instantiation)**. **ali** değişkeni referans tip olup kendisi yığın (**stack**) bellekte yer alır. Referansını tuttuğu nesneye öbek (**heap**) bellekte yer ayrılmıştır.

- ◇ 4. Satırda `ali` nesnesinin `yaş` durumu yani verisi `40` olarak değiştirildi. Bu durum değişimi `ali` nesnesinin davranışlarını etkiler!
- ◇ 5. Satırda `ali` nesnesine `YaşınıSöyle()` mesajı verildi (**message-passing**). Böylece `ali` nesnesi durumundan veri olarak davranışını konsola `Yaşım:40` yazarak sergiliyor.
- ◇ 6. Satırda `ali` nesnesine `CinsiyetiniSöyle()` mesajı verildi. Böylece `ali` nesnesi durumundan veri olarak davranışını konsola `Cinsiyetimi Söylemek İstemiyorum!` yazarak sergiliyor. Çünkü `ali` nesnesi imal edilirken cinsiyetine varsayılan olarak bir değer atanmamıştır!
- ◇ 8. ve 24. Satırlar arasında **kullanıcı tanımlı Öğrenci** sınıfı tanımlanmıştır.
 - 8. Satırda kullanıcı tanımlı **sınıf kimliği Öğrenci** olarak belirlenmiştir. Sınıflar kod bloğu olmadan tanımlanamaz. 9. Satırda kod bloğu açılmış, 30. Satırda kod bloğu kapatılmıştır. Sınıf üyeleri bu blok içinde tanımlanmıştır.
 - 11. Satırda sınıftan imal edilecek nesnelerin yaşına ait durumları tutacak **yaş** kimlikli **alan değişkeni** (**field variable**) tanımlanmıştır. Başındaki **public** sıfatı, imal edilecek nesnelerin bu verisinin umuma açık olacağını belirtir.
 - Benzer şekilde 12. Satırda sınıftan imal edilecek nesnelerin cinsiyetine ait durumları tutacak **cinsiyet** alan değişkeni tanımlanmıştır.
 - 14. ve 18. Satırlar arasında imal edilecek nesnelerin `YaşınıSöyle()` kimlikli **yöntem** (**method**) tanımlanmıştır. Başındaki **public** sıfatı, imal edilecek nesnelerin bu verisinin umuma açık olacağını belirtir.
 - Benzer şekilde 19. ve 28. Satırlar arasında imal edilecek nesnelerin `CinsiyetiniSöyle()` kimlikli yöntem tanımlanmıştır.

- ◇ Sınıftan nesne imal etme işlemine **örnekleme** (**instantiation**) adı verilir.
- ◇ Her bir nesne, sınıfın bir örneğidir (**instance**).
- ◇ Örnek üyelerine programın 4., 5. ve 6. satırlarında üzere **nokta işleci** (**dot operator**) (`.`) ile erişilir.

8.3 Erişim Değiştiricileri

Sınıf üyelerinin (alan değişkeni veya yöntem) başka nesne ve sınıfların nasıl erişeceğini **erişim değiştiricileri** (**access modifier**) belirler. **Görünürlük** (**visibility**) sıfatları olarak da adlandırılan bu değiştiriciler, sınıf üyelerine sınıf içinden ve dışından erişimi belirleyen sıfatlardır.

- ◇ **public**: Sınıf üyesine, sınıf dışından her zaman erişilebilir. **Umuma açık** (**public**) davranış ve özellikleri için bu değiştirici kullanılır.
- ◇ **private**: Sınıf üyesine, sınıf dışından erişime hiçbir zaman izin verilmez. **Mahrem/şahsi/kişisel** davranış ve özellikleri için bu değiştirici kullanılır. Varsayılan (**default**) erişim değiştiricidir. Bir sınıfta hiçbir erişim değiştirici yoksa tanımlanan durum ve davranışlar **mahrem** (**private**) olarak kabul edilir.
- ◇ **protected**: Dış dünyaya kapalı (**private**), ama mirasçılara açık sınıf üyeleri için **korunmalı** (**protected**) bu değiştirici kullanılır. Miras alma başlığında anlatılacaktır.
- ◇ **internal**: Sınıf üyesine, içinde bulunduğu proje ile kısıtlı olacak şekilde sınıf dışından her zaman erişilebilir.
- ◇ **file**: Sınıf üyesine, içinde bulunduğu kaynak dosyası ile kısıtlı olacak şekilde sınıf dışından her zaman erişilebilir.

Şimdiye kadarki örnekler de **tek bir konsol projesi** (**project**) üzerinden örnekler verilmiştir. Bazı durumlarda geliştireceğimiz uygulama **birden fazla projenin olduğu bir çözüm** (**solution**) olabilir. Bu durumda **protected internal**, **private protected** gibi erişim belirleyiciler de ihtiyaca göre kullanılabilir. Aşağıda tüm erişim değiştiricilerin bir tablosu verilmiştir. Yeri geldikçe bunlara örnek verilecektir.

Aşağıda erişim belirleyicilerin uygulandığı basit bir **Kişi** sınıfı örnek verilmiştir. Bu sınıftan imal edilen nesnelerin `yaş` durumu diğer tüm nesneler tarafından erişilip değiştirilebilir, ancak `sır` durumuna ise hiçbir nesne tarafından erişilemez. `maas` durumuna ise **Kişi** sınıfı ve bu sınıftan türemiş sınıflardan imal edilen nesneler tarafından bu özelliğe erişilebilir. İleride göreceğiz.

```
//ANA PROGRAM:
Kişi ahmet = new Kişi();
ahmet.adı = "Ahmet";
ahmet.AdınıSöyle();
//ali._sır="Benim Sırrım!"; /* Alan değiştirilemez! Çünkü mahrem (private) olarak tanımlanmış. */
ahmet.SırDeğiştir(); //Nesne, kendi içinde sırrını değiştirebilir.
```

Erişim Değiştiricisi	Sınıf İçi	Aynı Proje	Miras Alan	Dış Proje
public	✓	✓	✓	✓
private	✓	×	×	×
internal	✓	✓	✓*	×
protected	✓	×	✓	✓*
protected internal	✓	✓	✓	✓*
private protected	✓	✓*	✓*	×

* Not: Miras alma durumunda, belirleyicinin proje kısıtlamaları geçerliliğini korur.

Tablo 8.1: Erişim Belirleyicileri ve Erişim Sınırları

```
//SINIF:
class Kişi // "Kişi" Kimlikli Sınıf
{
    // Alanlar (Fields)
    public string adı;
    private string _sır; //mahrem(private) alanlar önünde altçizgi karakteri olacak şekilde küçük
    ↪ harfle kimliklendirilir
    // Yöntemler (Methods)
    public void SırDeğiştir()
    {
        _sır = "Gizli bir şey öğrendim!";
    }
    public void AdınıSöyle()
    {
        Console.WriteLine($"Adım:{adı}");
    }
}
```

8.4 Yöntemler

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı 2.7 başlığında anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız.

Nesne yönelimli programlamada ise birbiriyle ilgili nesnelere ilişkin sınıflar bir araya toplanarak bir sınıf kütüphanesinde tutulur. Bu durum 5.1 başlığı altında anlatılmıştı.

Bir bilgisayar programı, genel olarak, tek başına tüm görevleri yerine getiren tek bir fonksiyonundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre bu fonksiyon bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani **böl ve yönet (divide and conquer)** tekniği uygulanır. Çünkü **büyük bir problemin toptan çözülmesi**, problemin **parçalar halinde çözülmesinden her zaman daha yorucu ve zordur**. Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna **yukarıdan aşağıya (top down)** yaklaşım adı verilir.

Nesnelerin davranışlarını yöntemlerle tanımlarız. Yöntemler de yapısal programlamadaki fonksiyonlara karşılık gelir;

- Yöntemler, nesnenin belli bir davranışını gerçekleştiren bir kod bloğudur. Bu nedenle, **sınıf (class)** ve ileride göreceğimiz **yapı (struct)** ve **arayüz (interface)** içinde tanımlanır.
- Yöntemler parametrelili alabilir, girdilerle veya nesnenin durumlarıyla parametresiz olarak bir şeyler yapar ve ardından cevabını verebilir. Nesnede olmayan bilgiyi yöntemlere argümanlar ile aktarırız.
- Sınıf içinde her yöntemin bir kimliği vardır. Kimliklendirme **PascalCase** olarak (sözcüklerin baş harfler büyük olacak şekilde bitişik yazılarak) yapılır. Kimliklendirme kuralları için 4.2.5 başlığındaki değişken kimliklendirme kurallarına uyulur.
- Bir yöntem, çağıran koda bir değer döndürebilir.
- Eğer birden fazla davranış içeriyorsa hepsi ayrı tanımlanmalıdır.
- Bir fonksiyon, bir başka fonksiyondan çağrılabilir. Bir başka deyişle yöntemler tekrar kullanılabilir kodlardır.

§8.4.1 Yöntem Tanımı Nasıl Yapılır

Yöntem tanımı (method definition), yöntemin işleyip **döndüreceği veri tipini**, **kimliği**, işleyeceği bağımsız değişkenleri temsil eden **parametre listesini**, içerecek şekilde **başlığının** ve **gövdesinin** kodlanmasından oluşur.

```
class Sınıf {
    Erişim-Değiştirici Seçimlik-Erişim-Değiştirici Dönüş-tipi YöntemKimliği ( //Parametre Listesi:
        [veritipi parametre1]
        [,veritipi parametre2]
        [,veritipi parametre3]...
    )
    { //Yöntem Blok Başlangıcı
        /* Yöntem Gövdesi ... */
        [return dönüş-tipinde-değer;]
        /* Yöntem Gövdesi ... */
    } //Yöntem Bloğu Bitişi
}
```

Sözde kodu şu şekilde açıklayabiliriz;

- ◊ Alan değişkenlerinde olduğu gibi yöntemlerinde başında erişim değiştiricileri bulunur.
- ◊ Seçimlik erişim değiştiricileri olan **static**, **override**, **virtual**, **abstract**, **new** ve **sealed** olabilir. İleride anlatılacaktır.
- ◊ **Yöntemin kimliği benzersizdir**. Aynı zamanda **yöntem adı (method name)** olarak adlandırılır. Yöntem kimliklendirmede, 4.2.5 başlığındaki değişken kimliklendirme kuralları geçerlidir.
- ◊ Yöntem kimliğinden sonra **parametreleri barındıracak açıp kapatılan parantez** olmalıdır. Bu durum değişken tanımından çağrılabilir yöntem tanımlarını ayırmak içindir.
- ◊ Bağımsız değişkenler, parametre olarak adlandırılır;
 - **Parametre listesi**, bir fonksiyonun parametrelerinin veri tipini, sırasını ve sayısını belirtir.
 - Parametreler isteğe bağlıdır, yani bir yöntem hiçbir parametre içermeyebilir.
 - **Her bir parametreye kimlik verilmelidir**. Çünkü yöntem gövdesinde bu değişkenler işlenir.
 - Parametreler birbirinden **virgül işleci (comma operator) (,)** ile ayrılır.
- ◊ Parametreler yöntem **gövdesinde** yapılacak işlere temel oluşturur;
 - Gövde, yöntemin ne yaptığını tanımlayan ve süslü parantezler arasında yer alan talimatları içerir.
 - Gövdede, **yapısal programlama yapıldığından**, ilk önce olacaksa yerel değişkenler tanımlanmalı daha sonra parametrelerle birlikte bu değişkenleri işleyecek talimatlar yazılmalıdır.
 - **Yerel değişkenler (local variable)**, yöntem gövdesi içerisinde tanımlanan değişkenlerdir. Yöntemden çıkılınca kaybolurlar.
 - Gövdede, yöntemde istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani **algoritması belirlenir**.
- ◊ İşlenen parametrelerden, alan değişkenlerinden ve gövde içerisinde tanımlanan değişkenlerden bir **geri dönüş değeri** hesaplanabilir. Değer hesaplanmış ise **return geri-dönüş-değeri**; talimatıyla yöntemin icrası sonlandırılır. `textttreturn` Talimatı çoğunlukla fonksiyonundaki son talimat olarak görünür ama yöntemin gövdesinde herhangi bir yerden geri dönülebilir.
- ◊ **return** Talimatıyla geri döndürülen değer tipi, **yöntemin geri dönüş tipidir**.
- ◊ C# programlama dilinde **hiçbir değeri olmayan ve bellekte yer kaplamayan void** anahtar kelimesi vardır. **void**, bir yöntemin değer döndürmediğini belirtir. Yani yöntemin **işini yaptığını** ancak **çağırılan yere herhangi bir veri teslim etmediğini** ifade eder. Bu durumlarda fonksiyon, geri değer döndürmeyebilir;
 - Bu durumda yöntemin **geri dönüş tipi yoktur** ve bu nedenle geri dönüş tipi **void** olarak belirtilir.
 - Geri dönüş tipi **void** olan yöntemin icrası **return**; talimatıyla sonlandırılır.
 - Eğer **return**; talimatı hiç yazılmaz ise süslü parantez kapatılıncaya kadar fonksiyon icra edilir ve fonksiyondan geri dönülür.

Bir de yöntemin çağrıldığı yerde, **yöntemin döndürdüğü değer**in **tipi ile eşleşen bir değişkene dönüş değeri atanabilir**. Zorunlu değildir.

Ayrıca bir yöntem gövdesinde bir başka yöntem tanımlanamaz!

C# dilinde **void** kelimesinin kullanımı, C++ dilindeki **void** mantığıyla tamamen aynıdır. Ancak C#'ta **void** kullanımı daha güvenlidir (**managed code**).

C# dilinde yöntemler, yapısal programlama dilindeki fonksiyonlar gibi tanımlanır. 2.7 başlığında anlatılan yapısal programlamada veri yapıları (**data structure**) ve kontrol yapıları (**control structure**) birbirinden ayrıdır. Yapısal programlamadaki gibi yöntemler kodlanırken, önce veriyi taşıyan veri yapıları tanımlanmalı daha sonra veriyi işleyecek kontrol yapıları kullanılmalıdır.

§8.4.2 Nesneye Mesaj Verme

Bir nesnenin davranış göstermesini istiyorsak ona mesaj göndeririz (**message-passing**). Bunun yapısal programlamadan gelen adı **çağrıdır** (**call/invoke**).

Aşağıdaki örnekte, **Hesap** sınıfının **Topla()** ve **ÖrnekHesap()** yöntemlerini inceleyebilirsiniz. Bu sınıftan imal edilen **hesap** nesnesine davranış göstermesi için çeşitli mesajlar verilmiştir;

```
//ANA PROGRAM:
Hesap hesap = new Hesap();
Console.WriteLine(hesap.Topla(20, 30)); /* hesap nesnesine Topla mesajı verildi. Topla yönteminin
→ geri döndürdüğü değer konsola yazdırıldı. */
hesap.ÖrnekHesap(); /* hesap nesnesine ÖrnekHesap mesajı verildi. ÖrnekHesap yöntemi geri değer
→ döndürmüyor! */
Console.WriteLine(hesap.KatsayıylaÇarp(11)); /* hesap nesnesine KatsayıylaÇarp mesajı verildi.
→ KatsayıylaÇarp yönteminin geri döndürdüğü değer konsola yazdırıldı. */

//SINIF
class Hesap // Hesap Sınıfı
{
    private int _çarpan;
    public Hesap()
    {
        this._çarpan=10;
    }
    public int Topla(int p1, int p2)
    {
        return p1 + p2;
    }
    public void ÖrnekHesap()
    {
        Topla(1, 2); /* Topla yöntemi, 1 ve 2 argümanlarıyla çağrılıyor. Ama geri döndürdüğü
        → değer kullanılmıyor */

        Console.WriteLine(Topla(3, 4)); /* 3 ve 4 argümanlarıyla çağrılan Topla yönteminin geri
        → döndürdüğü değer konsola yazdırıldı */

        int toplam; //toplam yerel değişkeni tanımlandı
        toplam = Topla(5, 6); /* 5 ve 6 argümanlarıyla çağrılan Topla yönteminin geri döndürdüğü
        → değer toplam değişkenine artandı */
        Console.WriteLine(toplam);

        Console.WriteLine(Topla(7, 8) + Topla(9, 10)); /* Topla yönteminin geri döndürdüğü değer
        → ile bir başka Topla yönteminin geri döndürdüğü değer toplanarak konsola yazdırıldı */

        Console.WriteLine(Topla(10, Topla(11, 12))); /* Topla yönteminin argüman olarak bir başka
        → Topla yönteminin geri döndürdüğü değer geçirildi ve konsola yazdırıldı */
    }
    public int KatsayıylaÇarp(int p1)
    {
        return this._çarpan*p1; // alan değişkeni yöntemde kullanılıyor.
    }
}
```

§8.4.3 Yöntemlere Değerle Parametre Geçirme

Bir yönteme değer tip parametre olarak geçirildiğinde çağrı sırasında bellekteki değişim aşağıdaki programdan anlaşılabilir;


```
//Ana program değişkenleri:
```

```
int a = 10;
int b = 20;
int toplam = 0;
Hesap hesap = new Hesap();
```

```
//ÇAĞRI ORTAMI:Ana program nesnenin yöntemlerini çağırarak.
```

```
Console.WriteLine($"Çağrı öncesi a:{a}, b:{b}"); //çağrı öncesi a:10, b:20
```

```
toplam = hesap.Topla(a, b); //Değer tiplerin yöntemle geçirilmesi
```

```
Console.WriteLine($"Çağrı sonrası a:{a}, b:{b}, toplam:{toplam}"); //çağrı sonrası a:10, b:20,
→ toplam:30
```

```
//SINIF:
```

```
public class Hesap
```

```
{
```

```
    public int Topla(int x, int y)
```

```
    {
```

```
        int temp = x + y;
```

```
        x = 100; y = 200; // Parametre değişkenleri değiştiriliyor!
```

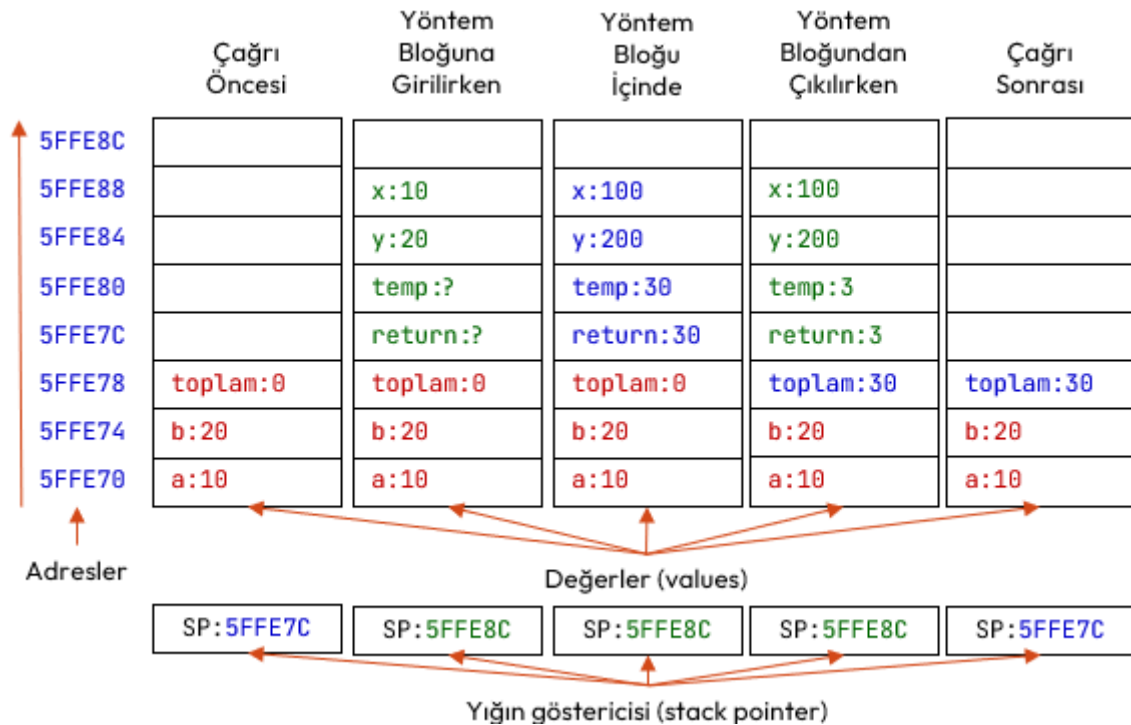
```
        Console.WriteLine($"Çağrı içinde x:{x}, y:{y}"); //çağrı içinde x:100, y:200
```

```
        return temp;
```

```
    }
```

```
}
```

Yukarıda örneği verilen programda `hesap` nesnesinin `Topla()` yönteminin çağırma sürecindeki yığın bellek bölgesindeki (stack segment) değişimi Şekil 8.1’de gösterilmektedir.



Şekil 8.1: Yöntem Çağrısı Sırasında Yığın Bellek

Yukarıdaki kod şu şekilde açıklanabilir;

- İcra sırası `hesap.Topla(a,b)` talimatına geldiğinde yöntem çağrılmadan önce ana programda yerel değişkenler `a`, `b` ve `toplam` yığın belleğe (stack segment) itilmiştir (push).
- Sonra `hesap.Topla(a,b)` yöntemi çağrıldığı anda `a` ve `b` değişkenlerinin değerleri, `x` ve `y` parametrelerine kopyalanarak fonksiyona geçirilir. Değerler parametrelere kopyalandığından buna **değerle çağırma** (call by value) adı verilir.
- `hesap.Topla(a,b)` yöntemi icra edilirken, yereldeki `temp` ve parametre değişkenleri `x`, ve `y` istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- `hesap.Topla(a,b)` yöntemi içine bir başka yöntem de çağrılabilir. Böyle bir durumda `x`, `y` ve `temp` değerleri yığın belleğe itilir ve çağrılan yöntem icra edilir. Böylece iç içe yapılan her çağrıda yığın

belleğin büyüyeceği görülmektedir. Program derlendiğinde yığın bellek (**stack segment**) için sınırlı bir bellek bölgesi ayrılır. İç içe yapılan çağrılarda bu bellek sınırı aşılabılır. Bu durumda program, **yığın taşması (stack overflow)** hatası verecektir.

- ◊ `hesap.Toplam=topla(a,b)` fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki `toplama` değişkenine kopyalanır ve ana programda yığın belleğe itilmiş değerler geri çekilir (**pop**).

§8.4.4 Yöntemlere Referansla Parametre Geçirme

Referans değişkenler, değerin kendisine değil bellek konumuna bir referanstır. Referans değişkenlerin işaret ettiği adresler yığın (stack) bellekte tutulur. Bir yonteme referans veri tipleri ya da bir **referansı parametre olarak geçirdiğinizde (pass by reference)**, referansların işaret ettiği adreslerin kopyaları parametre olarak kopyalanır. Parametrelerin değerleri için yeni bir depolama konumu oluşturulmaz. Böylece orijinal değişkenleri değiştirme imkânımız olur. Ayrıca hafızada çok yer kaplayan parametreler için çok kullanışlıdır.

Değer tipindeki parametreleri yöntem içerisinde değiştirmek istiyorsak önünde **ref** sıfatını (**ref keyword**) kullanmalıyız.

```
Hesap işlemci = new Hesap();
int a = 10;
int b = -25;
Console.WriteLine($"Çağrı öncesi a:{a}, b:{b}"); // a:10 b:-25
işlemci.DeğerDeğiştir(ref a, ref b);
Console.WriteLine($"Çağrı sonrası a:{a}, b:{b}"); // a:-25 b:10
```

```
class Hesap
{
    public void DeğerDeğiştir(ref int x, ref int y)
    {
        int temp= x;
        x = y;
        y = temp;
    }
}
```

§8.4.5 Yöntem Parametrelerini Çıktı Olarak Kullanma

Bir yöntemin içinden bir değeri dışarı çıkarmak için **return** talimatı kullanılır. Ancak bu talimat bir yöntemden yalnızca bir değer döndürmek için kullanılabilir.

Bunun yanında **out** parametrelerini kullanarak bir fonksiyondan birden fazla değer döndürebilirsiniz. **out** Parametreleri, referans parametrelerine benzerdir, ancak veriyi yonteme aktarmak yerine yöntemden dışarı aktarırlar.

```
Hesap hesap = new Hesap();
int okunan, okunanınKaresi;
okunan = hesap.birSayıOku(out okunanınKaresi);
Console.WriteLine($"Okunan: {okunan}, karesi:{okunanınKaresi}");
```

```
class Hesap
{
    public int birSayıOku(out int kare)
    {
        int temp;
        Console.WriteLine("Bir sayı giriniz: ");
        temp = Convert.ToInt32(Console.ReadLine());
        kare = temp * temp;
        return temp;
    }
}
/*Program Çıktısı:
Bir sayı giriniz:
12
Okunan: 12, karesi:144
*/
```


§8.4.6 Varsayılan Argüman

Argümanlar, yöntemin çağrıldığı anda yönteme geçirilen gerçek parametrelerdir. Aşağıda verilen **Topla** yönteminin iki farklı **int** parametresi aldığı görülmektedir. C#4.0+ ile gelen özellikle istenirse bu parametrelere varsayılan bir değer atanabilir. Yani yöntemler **varsayılan argümanlar (default argument)** ile tanımlanabilir.

```
Hesap işlem = new Hesap();
int sonuç= işlem.Topla(10); // 10 + 1 = 11
Console.WriteLine($"Sonuç: {sonuç}"); //11

class Hesap
{
    public int Topla(int p1, int p2 = 1) //İkinci argüman varsayılanı 1
    {
        return p1 + p2;
    }
}
```

§8.4.7 Özyinelemeli Yöntemler

Özyineleme (recursion), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, yineleme (**iteration**), döngüler (**loop**) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. Özyinelemeli algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır;

1. Cevabı kesin olarak bildiğimiz temel durum (**base case**)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n sayısının faktöriyeli, $n!$ ile gösterilir ve n sayısına eşit ve küçük olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 3 \times 2 \times 1$$

Sıfır sayısı da pozitif bir sayıdır ama sonucun sıfır çıkmaması için **sıfır sayısının faktöriyeli 1 kabul edilir**. Bu aslında faktöriyel problemi için **temel durumdur (base case)** ve **boş ürün (empty product)** kuralı olarak bilinir. Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
Fakyöriyel f = new Fakyöriyel();
Console.WriteLine(f.FaktöriyelHesapla(5)); // 120
```

```
class Fakyöriyel
{
    public uint FaktöriyelHesapla(uint n)
    {
        uint sonuc = 1, sayaç;
        for (sayaç = n; sayaç > 0; sayaç--)
            sonuc = sonuc * sayaç;
        return sonuc;
    }
}
```

Aslında negatif olmayan bir tam sayı olan sayının faktöriyeli, kendisi ile kendisinden bir küçük olan sayının faktöriyeli ile çarpımı şeklinde yazılabilir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \times (n - 1)!$$

Bu durumda faktöriyel fonksiyonu aşağıdaki şekilde **özyinelemeli (recursive)** tanımlanabilir;

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n - 1)! & \text{eğer } n > 0 \end{cases}$$

Aynı faktöriyel fonksiyonu **parametrelili olarak** aşağıdaki şekilde yine özyinelemeli olarak tanımlanabilir;

$$f(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot f(n - 1) & \text{eğer } n > 0 \end{cases}$$

Buna örnek olarak aşağıdaki hesabı verebiliriz;

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! = 5 \times 4 \times 3 \times 2 \times 1 \times 0! = 120$$

Özyinelemeli (recursive) olarak ise faktöriyel yöntemi aşağıdaki şekilde kodlanabilir;

```
public uint Faktöriyel(uint n)
{
    return n == 0 ? 1 : n * Faktöriyel(n - 1);
}
```

Burada iç içe çağrılan her yöntemde **yığın (stack)** belleğin şiştiği unutulmamalıdır!

Bir başka örnek olarak, özyinelemeli Fibonacci sayısı hesaplama örneği verilebilir;

```
public int fibonacci(int i)
{
    if(i <= 2) return 1;
    return fibonacci(i - 2) + fib(i - 1);
}
```

Yine bir başka örnek, özyinelemeli olarak bir tam sayının kuvveti aşağıdaki gibi hesaplanabilir;

```
public int kuvvet(int taban, int üs)
{
    if (taban == 0) return 0;
    if (üs == 0) return 1;
    return taban * kuvvet(taban, üs - 1);
}
```

§8.4.8 Yöntem Parametresi Olarak Diziler

Yöntemlere parametre olarak gönderilen dizinin boyutu, köşeli parantez içindeki **virgül işlecinden (comma operator)** anlaşılır.

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```
int[] dizi = { 1, 2, 3, 4, 5 };
int[,] matris = new int[4, 4]{ { 1, 2, 3, 4},
                               { 5, 6, 7, 8},
                               { 9, 10, 11, 12},
                               {13, 14, 15, 16} };

Diziİşlemleri işlemler = new Diziİşlemleri();
işlemler.diziyaz(dizi);
/*
Dizi:
1  2  3  4  5
*/
Console.WriteLine("Ortalama: {0}", işlemler.ortalama(dizi)); // Ortalama: 3
işlemler.matrisyaz(matris);
/*
Matris:
1  2  3  4
5  6  7  8
9  10 11 12
13 14 15 16
*/
işlemler.transpose(matris);
işlemler.matrisyaz(matris);
/*
Matris:
1  5  9 13
2  6 10 14
3  7 11 15
4  8 12 16
*/

class Diziİşlemleri
```

```

{
    public float ortalama(int[] dizi)
    {
        return (float)dizi.Average();
    }
    public void diziyaz(int[] dizi)
    {
        Console.WriteLine("Dizi: ");
        for (int i = 0; i < dizi.Length; i++)
            Console.Write($"{dizi[i],4}");
        Console.WriteLine();
    }
    public void transpose(int[,] dizi)
    {
        for (int i = 0; i < dizi.GetLength(0); i++) {
            for (int j = i; j < dizi.GetLength(1); j++) {
                int temp = dizi[i, j];
                dizi[i, j] = dizi[j, i];
                dizi[j, i] = temp;
            }
        }
    }
    public void matrisyaz(int[,] dizi)
    {
        Console.WriteLine("Matris:");
        for (int i = 0; i < dizi.GetLength(0); i++){
            for (int j = 0; j < dizi.GetLength(1); j++){
                Console.Write($"{dizi[i, j],4}");
            }
            Console.WriteLine();
        }
        Console.WriteLine();
    }
}

```

§8.4.9 Yöntemlerin Aşırı Yüklenmesi

Yöntemlerin aşırı yüklenmesi (method overloading), aynı isimdeki bir metodun farklı parametre listeleriyle birden fazla kez tanımlanabilmesidir. C# derleyicisi, hangi yöntemin çağrılacağına karar verirken **yöntem imzasına (method signature)** bakar. Bu özellik, programcının benzer işleri yapan metotlar için farklı isimler türetme zahmetini ortadan kaldırır. Bir **yöntemin imzası** şunlardan oluşur:

1. Metodun ismi.
2. Parametrelerin sayısı.
3. Parametrelerin veri tipleri.
4. Parametrelerin sırası.
5. Metodun geri dönüş tipi (`int`, `void` vb.) ve erişim belirleyicileri (`public`, `private`) metodun imzasının bir parçası değildir. Sadece geri dönüş tipi farklı olan iki metot aşırı yüklenmiş sayılmaz ve derleme hatası verir!

Aşağıda `Topla()` yönteminin aşırı yüklenmesine ilişkin örnek verilmiştir;

```

public class Hesaplayici
{
    // 1. İki tamsayıyı toplar
    public int Topla(int sayi1, int sayi2)
    {
        return sayi1 + sayi2;
    }
    // 2. Üç tamsayıyı toplar (Parametre sayısı farklı)
    public int Topla(int sayi1, int sayi2, int sayi3)
    {
        return sayi1 + sayi2 + sayi3;
    }
    // 3. İki ondalıklı sayıyı toplar (Parametre tipi farklı)
    public double Topla(double sayi1, double sayi2)

```

```

{
    return sayi1 + sayi2;
}
}

```

Yukarıdaki kod incelendiğinde;

- Okunabilirlik: `ToplaInt`, `ToplaDouble`, `ToplaUcSayi` gibi karmaşık isimler yerine tek bir `Topla` ismiyle temiz bir kod yapısı sağlar.
- Esneklik: Kullanıcıya aynı işlemi farklı veri tipleriyle yapma özgürlüğü tanır.
- Kod Tamamlama Desteği: Visual Studio gibi editörlerde metod ismini yazdığınızda kod tamamlama (*intellisense*) aracı, aşırı yüklenmiş tüm versiyonlar "1 of 3" şeklinde listelenerek yazılımcıya rehberlik eder.

Aşırı yükleme yaparken `ref` ve `out` anahtar kelimeleri de imzanın bir parçası kabul edilir. Ancak bir metodu hem normal parametrelili hem de aynı tipte `ref` parametrelili olarak tanımlamak belirsizliğe yol açacağı için izin verilmez.

```

// Bu iki kullanım birlikte aşırı yükleme OLABİLİR:
public void Islem(int x) { }
public void Islem(string x) { }

// Ancak bu ikisi AYNI ANDA bulunamaz:
// public void Islem(int x) { }
// public void Islem(ref int x) { } // Hata!

```

§8.4.10 Çağrı Kuralı

Çağrı kuralı (*calling convention*), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya göre iletileceğini belirtir;

- Birincisinde argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. C# dilinde bu kullanılır.

```

Sayiİşlemleri işlem = new Sayiİşlemleri();
int sonuç = işlem.topla(10,20,30); //p1:10,p2:20,p3:30
Console.WriteLine($"Sonuç: {sonuç}"); // Sonuç: 60
int a = 1;
int toplam = işlem.topla(a, ++a, a++);
/* Beklenen Sonuç: 1+2+2=5 Parametreler: p1:1,p2:2,p3:2 */
Console.WriteLine($"Sonuç: {toplam}"); // Sonuç: 5
Console.WriteLine($"a'nın son değeri: {a}"); // a'nın son değeri: 3

```

```

class Sayiİşlemleri
{
    public int topla(int p1, int p2, int p3)
    {
        Console.WriteLine($"p1:{p1},p2:{p2},p3:{p3}");
        return p1 + p2+ p3;
    }
}

```

Yukarıdaki örnek incelendiğinde `sonuç1` ve `sonuc2` argümanların hangi sırada fonksiyona geçirildiğinin bir önemi yoktur.

- İkincisinde ise C/C++ dilinde kullanılır ve argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir.

```

#include <iostream>
using namespace std;
int topla(int, int, int);
int main () {
    int a = 1, toplam;
    toplam = topla(a, ++a, a++);
    cout << "toplam:" << toplam << endl;
}
int topla(int x, int y, int z) {
    cout << "x:"<< x << " y:" << y << " z:" << z<< endl;
    return x+y+z;
}
/* Program çalıştırıldığında:

```

```
x:3 y:3 z:1
toplam:7
*/
```

Bu kodun çıktısı 1 2 3 olarak beklenir ancak çağrı kuralından ötürü çıktı 3 3 1 olur. Bunun nedeni C++ dilinin çağrı kuralının sağdan sola olmasıdır. Bunu şöyle açıklayabiliriz;

- ◊ Fonksiyona sağdan ilk argüman olarak **a** değişkeni değeri 1 geçirilir.
- ◊ Ardından **a++** ifadesi ile **a** değişkeni 2'ye yükseltilir.
- ◊ Sonra **++a** ifadesi ile **a** değişkeni 3'e yükseltilir.
- ◊ Fonksiyona ikinci argüman olarak 3 geçirilir.
- ◊ Fonksiyona son olarak **a**'nın son değeri olan 3 geçirilir.

Çağrı kuralı, C++ dillerinde yazılmış bir kütüphanedeki fonksiyonları projemizde kullanmak istediğimizde önem kazanır¹.

8.5 Nesne Yapıcısı

Bir sınıftan birçok nesne de imal edilebileceği anlatılmıştı. İmal edilen her nesne (**object**) sınıfın bir örneğidir (**instance**) ve bu nesnelerin varsayılan (**default**) olarak farklı verilerle yani durumlarla (**state**) imal edilmesi gerekiyorsa **nesne yapıcısı** (**constructor**) kullanılır. Nesnelere ait veriler, alanlarda (**field**) tutulurlar. **Nesnenin durumları** (**state**), **nesnenin davranışını** (**behavior**) etkiler! Bu nedenle daha nesneler imal edilirken durumlarının belirlenen değerde olması istenir.

§8.5.1 Yapıcı Tanımlama

Yapıcılar tanımlanırken aşağıdaki kurallar uygulanır;

- ◊ **Yapıcıların kimliği, sınıf kimliğiyle aynıdır.**
- ◊ Yapıcıların görevi sınıftan nesneleri, varsayılan durumlara sahip olarak imal etmektir. Kısaca görevi, alan değişkenlerine ilk değer vermektir.
- ◊ Yapıcıların parametre listesi olabilir. Alan değişkenlerini atayan kod blokları vardır.
- ◊ Yapıcılar, yöntemlerin aksine **void** dahil **geri değer döndürmezler.**
- ◊ Bu nedenle bir yapıcı içerisinde **return** talimatı da kullanılmaz.

Aşağıda verilen örnekte **Öğrenci** sınıfından imal edilecek her nesnenin, yaşı 18 ve cinsiyeti 'E' olacaktır. **Hiçbir parametresi olmayan** bu yapıcıya **varsayılan yapıcı** (**default constructor**) adı verilir. Yapıcılar, **new** işleci ile nesne imal edilirken, nesnenin **verilerine/durumlarına ilk değer vermek için çağrılır**. Yani nesneler **başlatılırken** (**initialization**) çağrılırlar.

```
Öğrenci ilhan = new Öğrenci(); // Öğrenci() yapıcısı kullanılarak nesne imal edildi!
ilhan.YaşınıSöyle();           // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: ERKEK
```

```
class Öğrenci
{
    // Alanlar
    public uint yaş;
    public char cinsiyet;
    // 1. Varsayılan (Default) Yapıcı
    public Öğrenci()
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(cinsiyet))
        {
```

¹<https://learn.microsoft.com/en-us/dotnet/standard/native-interop/calling-conventions>

```

        case 'E':
            Console.WriteLine("Cinsiyetim: ERKEK");
            break;
        case 'K':
            Console.WriteLine("Cinsiyetim: KADIN");
            break;
        default:
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
            break;
    }
}
}

```

Eğer bir sınıfa hiç yapıcı yazmazsanız, C# derleyicisi **arka planda parametresiz ve boş bir yapıcı oluşturur**. Bunu 8.3 başlığındaki **Kişi** sınıfında görebiliriz. Ancak sınıfta herhangi bir yapıcı yazdığınız anda bu otomatik yapı kaybolur. Bunu da yukarıdaki örnekte görebilirsiniz. İki örnekte de nesne imalatı kısmını inceleyiniz.

§8.5.2 Yapıcı Aşırı Yükleme

Yöntemlerin aşırı yüklenmesinde, aynı kimlikle farklı imzaya (**signature**) sahip birden fazla yöntem tanımlaması yapılabilir. Benzer şekilde bir sınıfta **farklı parametre listelerine sahip** birden fazla yapıcı bulunabilir. Buna **yapıcı aşırı yükleme** (**constructor overloading**) adı verilir. Bu da, nesnenin farklı şekillerde başlatılmasına olanak tanır.

```

Öğrenci ilhan = new Öğrenci(); // Varsayılan Yapıcı İle Nesne İmalatı
ilhan.YaşınıSöyle(); // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle(); // Çıktı: Cinsiyetim: ERKEK
Öğrenci damla = new Öğrenci(20, 'K'); // Parametrelili Yapıcı İle Nesne İmalatı
damla.YaşınıSöyle(); // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle(); // Çıktı: Cinsiyetim: KADIN

```

```

class Öğrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcılar (constructors)
    public Öğrenci() // 1. Varsayılan (Default) Yapıcı
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    public Öğrenci(uint pYaş, char pCinsiyet) // 2. Parametrelili Yapıcı
    {
        yaş = pYaş;
        cinsiyet = pCinsiyet;
    }
    public Öğrenci(Öğrenci p) // 3. Parametrelili Kopya Yapıcı (copy constructor)
    {
        yaş = p.yaş;
        cinsiyet = p.cinsiyet;
    }

    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {

```



```

        switch (char.ToUpper(cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}

```

Yukarıda parametresi olmayan **varsayılan yapıcıya (default constructor)** ek olarak bir başka **parametrelili yapıcı (parameterized constructor)** tanımlanmıştır. Parametre listesi farklı olduğunda, **yapıcı aşırı yüklenmiştir (overloaded constructor)**.

§8.5.3 this Anahtar Kelimesi

this anahtar kelimesi (**this keyword**), nesne yönelimli programlamanın en temel **öz-referans** aracıdır. En basit tanımıyla **this**, o anda üzerinde çalışılan nesnenin yani örneğin kendisini temsil eder.

İsim karışıklığını giderme (**ambiguity**), en yaygın kullanım alanıdır. Yapıcılarda (**constructor**) veya yöntemlerde, parametre kimlikleri veya yerel değişkenleri sınıfın alan (field) değişkenleri ile aynı olduğunda **this** kelimesi kullanarak hangisinin sınıfa ait olduğunu belirtiriz.

```

Öğrenci damla = new Öğrenci(20, 'K'); // Parametrelili Yapıcı İle Nesne İmalatı
damla.YaşınıSöyle(); // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle(); // Çıktı: Cinsiyetim: KADIN

```

```

class Öğrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcılar (constructors)
    public Öğrenci(uint yaş, char cinsiyet)
    {
        this.yaş = yaş;
        this.cinsiyet = cinsiyet;
    }

    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}

```

}
Bazen bir yöntemin içindeyken, o anki nesneyi başka bir yöntem veya sınıfa parametre olarak göndermeniz gerekir. İşte burada "ben buradayım" demek için `this` kullanılır.

```
public class Veritabani
{
    public void Kaydet(Ogrenci ogr) { /* Kayıt işlemleri */ }
}
public class Ogrenci
{
    public void SistemeEkle()
    {
        Veritabani db = new Veritabani();
        // Bu sınıftan imal edilen nesnenin kendisini gönderiyoruz.
        db.Kaydet(this);
    }
}
```

Bu anahtar kelime sadece bu durumlarda kullanılmaz. Yöntemlerin içinde de kullanılır.

§8.5.4 Yapıcı Zincirleme

Bir yapıcının, aynı sınıf içindeki (veya üst sınıftaki) başka bir yapıcıyı kendi işini yapması için çağırması işlemine **yapıcı zincirleme** (**constructor chaining**) denir.

Bir yapıcı, başka bir yapıcıya **delege edilecekse**, yapıcı gövdesinden önce iki nokta üstüste işleci (**colon operator**) (`:`) ve `this` anahtar kelimesi kullanılır.

```
Ogrenci ilhan = new Ogrenci(); // Varsayılan Yapıcı İle Nesne İmalatı
ilhan.YaşınıSöyle();          // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle();     // Çıktı: Cinsiyetim: ERKEK
Ogrenci damla = new Ogrenci(20, 'K'); // Parametrelili Yapıcı İle Nesne İmalatı
damla.YaşınıSöyle();          // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle();     // Çıktı: Cinsiyetim: KADIN
Ogrenci reyhan = new Ogrenci('K'); // Delege Eden Yapıcı İle Nesne İmalatı
reyhan.YaşınıSöyle();          // Çıktı: Yaşım: 30
reyhan.CinsiyetiniSöyle();     // Çıktı: Cinsiyetim: KADIN
Ogrenci osman = new Ogrenci(55); // Delege Eden Yapıcı İle Nesne İmalatı
osman.YaşınıSöyle();           // Çıktı: Yaşım: 55
osman.CinsiyetiniSöyle();     // Çıktı: Cinsiyetim: ERKEK
```

```
class Ogrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcılar (constructors)
    public Ogrenci()// 1. Varsayılan (Default) Yapıcı
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    public Ogrenci(uint pYaş, char pCinsiyet)// 2. Parametrelili Yapıcı
    {
        yaş = pYaş;
        cinsiyet = pCinsiyet;
    }
    public Ogrenci(char pCinsiyet) : this(30,pCinsiyet)// 3. Delege Eden Yapıcı
    {
        // Delege Eden Yapıcı: Sadece cinsiyeti alanını ayarlar, yaşı varsayılan olarak 30 kabul
        // eder.
        // İş i Ogrenci(uint pYaş, char pCinsiyet) yapıcısına devreder (delegate).
    }
    public Ogrenci(uint pYaş) : this(pYaş, 'E')// 4. Delege Eden Bir Başka Yapıcı
```

```

{
    // Delege Eden Yapıcı: Sadece yaş alanını ayarlar, cinsiyeti varsayılan olarak 'E' kabul
    // eder.
    // İşi Öğrenci(uint pYaş, char pCinsiyet) yapıcısına devreder (delegate).
}
// Yöntemler
public void YaşınıSöyle()
{
    Console.WriteLine($"Yaşım: {yaş}");
}
public void CinsiyetiniSöyle()
{
    switch (char.ToUpper(cinsiyet))
    {
        case 'E':
            Console.WriteLine("Cinsiyetim: ERKEK");
            break;
        case 'K':
            Console.WriteLine("Cinsiyetim: KADIN");
            break;
        default:
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
            break;
    }
}
}

```

§8.5.5 Yapıcılarda Varsayılan Argümanlar

Yöntemlerde olduğu gibi aşırı yüklenmiş yapıcılarda da varsayılan argümanlar kullanılabilir. Bu durumda aşırı yüklemde parametre listesinin farklı olması kuralına uyulmalıdır.

```

Öğrenci ilhan = new Öğrenci(); // pYaş=18, pCinsiyet='E' Varsayılan Argümanı ile Nesne İmalatı
ilhan.YaşınıSöyle();           // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: ERKEK
Öğrenci abdullah = new Öğrenci(35); // Cinsiyet='E' Varsayılan Argümanı ile Nesne İmalatı
abdullah.YaşınıSöyle();        // Çıktı: Yaşım: 35
abdullah.CinsiyetiniSöyle();   // Çıktı: Cinsiyetim: ERKEK
Öğrenci damla = new Öğrenci(20, 'K'); // Başka Argümanlarla Nesne İmalatı
damla.YaşınıSöyle();          // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle();     // Çıktı: Cinsiyetim: KADIN

```

```

class Öğrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcı (constructor)
    public Öğrenci(uint pYaş=18, char pCinsiyet='E') // Varsayılan Argümanlarla Parametrelili Yapıcı
    {
        yaş = pYaş;
        cinsiyet = pCinsiyet;
    }

    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(cinsiyet))

```

```

    {
        case 'E':
            Console.WriteLine("Cinsiyetim: ERKEK");
            break;
        case 'K':
            Console.WriteLine("Cinsiyetim: KADIN");
            break;
        default:
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
            break;
    }
}
}

```

§8.5.6 Birincil Yapıcı

C#12+ ile gelen **birincil yapıcı** (**primary constructor**), yapıcı parametrelerinin sınıf gövdesinin her yerinde doğrudan kullanılabilmesini sağlar. Bu özellik, özellikle bağımlılık enjeksiyonu (**dependency injection**) kullanan sınıflarda kod karmaşasını ciddi oranda azaltır.

Geleneksel C# sınıflarında, dışarıdan gelen bir değeri sınıf içindeki özel bir alana (**field**) atamak için hem alanı tanımlamanız hem de yapıcı (**constructor**) içinde bu atamayı yapmanız gerekir. Birincil yapıcı, bu süreci doğrudan sınıf isminin yanına taşıyarak **bürokrasi** (**boilerplate**) oluşturan kod ihtiyacını ortadan kaldırır:

- ◊ **Doğrudan Kapsam:** Parametreler, sınıfın veya yapının tüm gövdesi boyunca doğrudan kullanılabilir.
- ◊ **Bürokrasi Azaltma:** Özel alan tanımlama ve yapıcı içinde `this.x = x` gibi tekrarlayan atama satırlarını yok eder.
- ◊ **Esneklik:** Yalnızca **record** yapılarında değil, artık standart **class** ve **struct** yapılarında da kullanılabilir.

```

// Geleneksel Yöntem
public class KitapServisiOld
{
    private readonly string _baglantiCumlesi;
    public KitapServisiOld(string baglantiCumlesi)
    {
        _baglantiCumlesi = baglantiCumlesi;
    }
}

// Modern Yöntem (Primary Constructor)
public class KitapServisi(string baglantiCumlesi)
{
    public void Baglan()
    {
        Console.WriteLine($"{baglantiCumlesi} kullanılarak bağlanıldı.");
    }
}

```

§8.5.7 required Anahtar Kelimesi

C#11+ ile hayatımıza giren **required** anahtar kelimesi, nesne yönelimli tasarımda "nesne bütünlüğünü" korumak için geliştirilmiş çok kritik bir araçtır. Bu özellik, bir sınıfın veya kaydın (**record**) belirli özelliklerinin nesne oluşturulurken mutlaka atanması gerektiğini derleyici düzeyinde garanti altına alır.

Modern C# yazımında nesne başlatıcıları (**object initializers**), yapıcılara (**constructor**) göre daha okunabilir bir alternatif sunar. Ancak nesne başlatıcılarının en büyük zayıflığı, bazı özelliklerin atanmasının unutulabilmesidir. **required** belirleyicisi (**required specifier**), bu boşluğu doldurarak hem esnekliği hem de veri güvenliğini aynı anda sağlar.

Eskiden bir özelliğin mutlaka doldurulmasını istiyorsak, onu sınıfa bir yapıcı parametresi olarak eklemek zorundaydık. Bu durum, çok fazla özelliği olan sınıflarda devasa ve yönetilmesi zor yapıcılara yol açıyordu. **required**, yapıcı yazma zorunluluğunu ortadan kaldırırken, derleyicinin o özelliğin boş geçilmesine izin vermemesini sağlar.

Bu lanlar internet formlarındaki kırmızı yıldızlı alanlar gibidir. Bilgileri istediğiniz sırayla girebilirsiniz ancak zorunlu bir alanı boş bıraktığınızda sistem (derleyici) işlemi tamamlamanıza izin vermez.

```
public class Personel
```

```
{
    public required int SicilNo { get; init; }
    public required string Müdürlük { get; set; }
}

// Derleme hatası verir: SicilNo eksik
// var p = new Personel { Müdürlük = "AR-GE" };
```

8.6 Soyut, Somut ve Bilgi Gizleme

Soyut (**abstract**) kavramı kelime itibarıyla detay içermeyen, özet anlamına gelir. Soyutlama (**abstraction**), bir nesnenin sadece "ne yaptığını" ortaya koyup, "nasıl yaptığını" gizleme sürecidir. Kullanıcıya sadece gerekli olan temel özellikleri sunar, karmaşık detayları arka planda tutar. Örnek olarak bir **Araba** sınıfı düşünün. Kullanıcı bu sınıftan imal edilmiş bir nesneden **gazaBas()** davranışını göstermesi için ileti gönderebilir. Motorun içindeki yanma odalarında neler olduğu (detay), kullanıcının bilmesi gereken bir şey değildir (soyutlama).

Soyut sınıflar (**abstract class**) da tanımlanabilir ancak bu sınıflardan nesne imal edilemez. Yani bu sınıfın bir **örneği** (**instance**) **olamaz!** Daha çok **kavramsal olarak kullanılır!** Soyut bir sınıfın en az bir soyut yöntemi (**abstract method**) olur. Soyut sınıflar hangi davranışı göstereceğini söyler ama nasıl davranış göstereceğini anlatmaz. Bu nedenle **abstract** kelimesiyle tanımlanan **soyut yöntem** (**abstract method**) için gövde tanımlanmaz. Aşağıda soyut bir sınıf tanımı verilmiştir;

```
//Soyut soyutnesne = new Soyut(); //HATA: Soyut sınıftan nesne imal edilemez!
```

```
abstract class Soyut
{
    public abstract void Yöntem(); //Soyut Yöntemler için gövde tanımlanmaz!
}
```

Somut (**concrete**) ise en ince detayına kadar bilinen, elle tutulur anlamına gelir. **Somutlama** (**concretization**), soyut olarak tanımlanmış bir yapının gerçek hayattaki karşılığının oluşturulmasıdır. **Soyut olmayan** geri kalan sınıflar **somut sınıftır** (**concrete class**) be bu sınıflardan nesne imal edilebilir. Yani bu sınıfların **örneği** (**instance**) vardır. İleride göreceğimiz bir arayüzün (**interface**) içinin doldurulması da somutlamadır. Yani kodlayarak uygulamanın hayata geçirilmesidir (**implementation**).

Bilgi gizleme (**information hiding**), bir nesnenin iç durumuna (verilerine) dışarıdan doğrudan erişilmesini engellemek ve bu verileri koruma altına almaktır. Genellikle ileride göreceğimiz kapsülleme (**encapsulation**) ile yapılır.

Bir sınıfta **soyutlama** (**abstraction**) iki türlü yapılır; Birincisi **veri soyutlaması** (**data abstraction**), nesnenin durumları üzerinde yapılan soyutlamadır. İkincisi ise **kontrol soyutlaması** (**control abstraction**), nesnenin davranışları üzerinde yapılan soyutlamadır.

8.7 Kapsülleme

Yukarıda verilen **Öğrenci** örneğinden devam edecek olursak imal edilecek nesnenin **yaş** durumuna bir başka nesne veya program, **-1** gibi bir değer atayabilir. Bu durumda **YaşınıSöyle()** davranışında istenmeyen bir çıktı verilmesine sebep olur. Bunu gidermek için **veri soyutlaması** (**data abstraction**) yapılması gerekir.

Veri soyutlaması bir alan değişkeni üzerinde şöyle yapılır;

- ◊ Öncelikle alan değişkenini mahrem (**private**) olarak tanımlayıp dışarıya erişime kapatmalıyız. Örnek: **private uint yaş;**
- ◊ Daha sonra alan değişkeniyle aynı olacak şekilde Büyük harfle başlayan bir umuma açık (**public**) özellik (**property**) tanımlanır. Örnek: **public uint Yaş { }**
- ◊ Sonra özelliğe alan değişkenine kontrollü değer atayacak **set** yöntemi ve alan değişkeninden değeri dışarıya aktaran **get** yöntemleri eklenir.

Öğrenci örneğine ilişkin sınıfımızın Yaş ve Cinsiyet özelliği eklenmiş hali aşağıda kodlanmıştır;

```
Öğrenci ilhan = new Öğrenci(30, 'E');
ilhan.Yaş = 50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
ilhan.YaşınıSöyle(); // Yaşım:50
ilhan.CinsiyetiniSöyle(); // Cinsiyetim:ERKEK
ilhan.Yaş = 5; /* ilhan nesnesinin Yaş özelliği değişmez: Öğrenci 6 Yaşından Küçük Olamaz.*/
ilhan.Cinsiyet = 'H'; /* ilhan nesnesinin Cinsiyet özelliği değişmez: Cinsiyet E,e,K,k,B,b
→ dışında bir değer olamaz. */
```

```

class Öğrenci
{
    //Alanlar
    private char _cinsiyet; //private alanlar altçizgi karakteriyle başlar
    private uint _yaş; //private alanlar altçizgi karakteriyle başlar
    //Yapıcı
    public Öğrenci(uint yaş,char cinsiyet)
    {
        this._yaş = yaş;
        this._cinsiyet = cinsiyet;
    }
    //Özellikler
    public uint Yaş //Yaş Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return _yaş; }
        set {
            if (value < 6)
                Console.WriteLine("Öğrenci 6 Yaşından Küçük Olamaz.");
            else
                _yaş = value;
        }
    }
    public char Cinsiyet //Cinsiyet Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return _cinsiyet; }
        set {
            string sonuc = value switch
            {
                'e' or 'E' => "ERKEK",
                'k' or 'K' => "KADIN",
                'b' or 'B' => "BELİRTMİYOR",
                _ => "X"
            };
            if (sonuc == "X")
                Console.WriteLine("Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz.");
            else
                _cinsiyet = value;
        }
    }
    //Yönetimler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{_yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        string sonuç = _cinsiyet switch
        {
            'e' or 'E' => "ERKEK",
            'k' or 'K' => "KADIN",
            _ => "SÖYLEMEK İSTEMİYORUM"
        };
        Console.WriteLine($"Cinsiyetim:{sonuç}");
    }
}

```

Nesneye ilişkin durum (**state**) verileri, alan değişkenlerinde saklanır. Bir durumun veri soyutlaması yapılmış haline **özellik** adı veririz. Çünkü artık bu duruma dışarıdan doğrudan **müdahale edilemez!** Nesnenin **durum üzerinde kontrol hakları vardır.**

Benzer şekilde davranışlar (**behavior**) da nesne dışına kontrollü olarak açılabilir. Bu durumda da **kontrol soyutlaması (control abstraction)** yapılır. Aşağıda faktöriyel hesaplayan bir hesap makinesinin ekranı

yenileme mahrem davranışı olan `EkraniYenile()` buna örnek verilebilir;

```
HesapMakinesi hesaplayici = new HesapMakinesi(); // Hesaplanan: 0
hesaplayici.Hesaplanan = 3;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 6
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 720 (6! = 720)
hesaplayici.MakineyiSıfırla();
hesaplayici.Hesaplanan = 8;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 40320 (8! = 40320)
```

```
class HesapMakinesi
{
    //Alan
    private int _hesaplanan;
    //Yapıcı
    public HesapMakinesi()
    {
        _hesaplanan = 0;
        EkraniYenile();
    }
    //Özellik
    public int Hesaplanan
    {
        set
        {
            _hesaplanan = value;
            EkraniYenile();
        }
        get
        {
            return _hesaplanan;
        }
    }
    //Yöntemler
    public void MakineyiSıfırla()
    {
        _hesaplanan = 0;
        EkraniYenile();
    }
    public void FaktöriyelHesapla()
    {
        int temp = 1;
        for (int sayac = _hesaplanan; sayac > 0; sayac--)
            temp = temp * sayac;
        _hesaplanan = temp;
        EkraniYenile();
    }
    private void EkraniYenile() //Mahrem Davranış!
    {
        Console.WriteLine($"Hesaplanan: {_hesaplanan}");
    }
}
```

Bir sınıf üzerinde veri soyutlaması (**data abstraction**) yapılarak veriler üzerine bir kılıf (**capsule**) çekilir, bir de kontrol soyutlaması (**control abstraction**) ile davranışların üzerine bir kılıf (**capsule**) daha çekilir. **Bir sınıf üzerinde her iki soyutlamanın yapılmasına kapsülleme (**encapsulation**) adı verilir.**

Soyutlamaların gerekli olup olmadığına programcı karar verir. Kapsülleme işlemi örneklerde görüldüğü üzere erişim değiştiricilerle (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış, istenmeyen detaylar dış dünyadan gizlenir (**information hiding**). Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden soyut (**abstract**) olarak kullanmak mümkün olmaktadır. Programcı, tasarladığı sınıfın soyut olarak kullanılacağını düşünerek sınıfları tasarlamalıdır.

Kapsülleme (**encapsulation**) tekniğini uygulamak, **kullanımı kolay** sınıf (**class**), bileşen (**component**), kütüphane (**library**) ve çerçeve yapılar (**framework**) elde etmemizi sağlar;

- ◊ n adet durum/veri (**state/data**) ve n adet davranış (**behavior**) kapsülleme işlemine tabi tutulursa sınıf,
- ◊ n adet sınıf kapsülleme işlemine tabi tutulursa bileşen,
- ◊ n adet bileşen kapsülleme işlemine tabi tutulursa kütüphane,
- ◊ n adet kütüphane kapsülleme işlemine tabi tutulursa çerçeve yapılar oluşur.

Sonuç olarak kapsülleme, projemizde proje süresini ve maliyetini kısaltma üstünlüğünü verir.

8.8 Nesne Başlatıcıları

Başlatma kavramı değişken tanımlama ile ilgili 4.2.3 başlığında anlatılmıştı. **Nesne başlatıcıları** (**object initializer**), bir nesnenin özelliklerine (**properties**) veya alanlarına (**fields**) nesne oluşturulma anında, açıkça bir yapıcı (**constructor**) çağırma gereke kalmadan değer atamamızı sağlayan bir söz dizimidir;

- ◊ **Okunabilirlik:** Yapıcı metotlara kıyasla daha okunabilir bir alternatif sunar. Parametrelerin neyi temsil ettiğini parantez içindeki sırasından değil, doğrudan isimlerinden anlarız.
- ◊ **Bürokrasi (Boilerplate) Azaltma:** Sadece veri taşımak için oluşturulan sınıflarda onlarca farklı yapıcı (**constructor overload**) yazma zorunluluğunu ortadan kaldırır.
- ◊ **required ile Tam Uyum:** C# 11 ile gelen **required** üyeler, nesne başlatıcıları aracılığıyla zorunlu hale getirilebilir.

Geleneksel yapıcılar, fabrikadan her şeyi monte edilmiş halde gelen bir masaya benzer. Masanın her detayı üretim bandında (**constructor**) belirlenmiştir ve sonradan değiştirmek zordur.

```
public class Kitap
{
    public string Ad { get; set; } = string.Empty;
    public string Yazar { get; set; } = string.Empty;
    public int SayfaSayisi { get; set; }
}

public class Program
{
    public static void Main()
    {
        // Geleneksel Yöntem (Constructor gerektirir)
        // var kitap1 = new Kitap("Suç ve Ceza", "Dostoyevski", 600);

        // Object Initializer Yöntemi
        var kitap2 = new Kitap
        {
            Ad = "Nutuk",
            Yazar = "Mustafa Kemal Atatürk",
            SayfaSayisi = 543
        };
    }
}
```

8.9 init Erişim Belirleyicisi ile Esnek Değişmezlik

C#9.0 ile tanıtılan **init** anahtar kelimesi, özelliklerin (**properties**) yalnızca nesne oluşturulma aşamasında atanabilmesini sağlayan bir erişim belirleyicidir. Bu yapı, nesne yönelimli tasarımda "değişmezlik" (**immutability**) ilkesini korurken, nesne başlatıcılarının (**object initializers**) sunduğu esneklikten ödün vermememizi sağlar.

Geleneksel **get; set;** yapısında özellikler nesne ömrü boyunca her an değiştirilebilir. Eğer salt okunur bir yapı istenirse **get; private set;** veya sadece **get;** kullanılabilir; ancak bu durumda değerlerin mutlaka bir yapıcı (**constructor**) üzerinden geçilmesi gerekir. **init** ise bu iki yaklaşımın ortasında durur: Nesne oluşturulurken (**new** anahtar kelimesi ile) değer atanmasına izin verir, ancak atama işlemi bittiği anda özelliği (**property**) salt okunur hale getirir.

Bir kargo kutusunu hazırladığınızı düşünelim. Kutunun içine eşyayı koyma ve üzerine adres etiketini yapıştırma yetkiniz vardır; ancak bu yetki sadece kutu kargoya verilmek üzere "hazırlanırken" geçerlidir. Kutu bir kez mühürlenip yola çıktıktan (nesne oluşturulduktan) sonra içindeki eşyayı değiştiremez veya etiketi

sökemezsiniz. `init`, o mühür vurulmadan önceki son atama yetkisidir.

```
public class Bilgisayar
{
    // Sadece başlangıçta atanabilir
    public string Islemci { get; init; }
    public int Ram { get; init; }

    // Her zaman güncellenebilir
    public decimal Fiyat { get; set; }
}

public class Program
{
    public static void Main()
    {
        // Nesne başlatıcı kullanarak atama yapmak mümkün
        var pc = new Bilgisayar
        {
            Islemci = "i9",
            Ram = 32,
            Fiyat = 45000m
        };

        // HATA: 'Islemci' init-only olduğu için sonradan değiştirilemez
        // pc.Islemci = "i7";

        // 'Fiyat' set olduğu için değiştirilebilir
        pc.Fiyat = 42000m;
    }
}
```

Bir başka örnek olarak dağıtıma çıkan bir kargo paketi örnek verilebilir. Bir paket hazırlanırken içeriği belirlenebilir ancak paket mühürlenip dağıtıma çıktığında içerik artık değiştirilemez. `init`, nesnenin mühürlenme anına kadar olan atama yetkisidir.

```
public class SensorVerisi
{
    public DateTime KayitZamani { get; init; }
    public double Deger { get; init; }
}

// Geçerli kullanım
var veri = new SensorVerisi { KayitZamani = DateTime.Now, Deger = 24.5 };
// Derleme zamanı hatası:
// veri.Deger = 25.0;
```

8.10 Miras Alma

Öğrenci, öğretmen ve müdürün olduğu bir okul sistemini örnek alarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir;

```
//ANA PROGRAM:
Öğrenci ali = new Öğrenci(20, 'E', 12000);
ali.YaşınıSöyle(); //Yaşım:20
ali.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK
ali.DersÇalış(); //Dersimi Çalışıyorum...

Öğretmen ilhan = new Öğretmen(50, 'E', 101);
ilhan.YaşınıSöyle(); //Yaşım:50
ilhan.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK
ilhan.DersVer(); //101 Dersini Veriyorum...

Müdür damla = new Müdür(30, 'K', 5000);
damla.YaşınıSöyle(); // Yaşım:30
damla.CinsiyetiniSöyle(); //Cinsiyetim: KADIN
```

```
damla.DersProgramıHazırla(); //5000 Bölümünde Ders Programı Hazırlıyorum...
```

```
//SINIFLAR:
```

```
class Öğrenci
```

```
{
    //yapıcı:
    public Öğrenci(uint pYaş, char pCinsiyet, uint pOkulNumarası) {
        Yaş=pYaş; Cinsiyet=pCinsiyet; OkulNumarası=pOkulNumarası;
    }
    //Özellikler:
    public uint Yaş { get; set; } //sanki "yaş" alan değişkeni varmış gibi kısa özellik tanımı!
    public char Cinsiyet { get; set; }
    public uint OkulNumarası { get; set; }
    // Yöntemler:
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
    public void DersÇalış()
    {
        Console.WriteLine("Dersimi Çalışıyorum...");
    }
}
```

```
class Öğretmen
```

```
{
    //Yapıcı:
    public Öğretmen(uint pYaş, char pCinsiyet, uint pVerdiğiDersKodu) {
        Yaş=pYaş; Cinsiyet=pCinsiyet; VerdiğiDersKodu=pVerdiğiDersKodu;
    }
    //Özellikler:
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public uint VerdiğiDersKodu { get; set; }
    // Yöntemler:
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
        }
    }
}
```

```

        break;
        default:
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
            break;
    }
}
public void DersVer()
{
    Console.WriteLine($"{VerdiğiDersKodu} Dersini Veriyorum...");
}
}
class Müdür
{
    //Yapıcı:
    public Müdür(uint pYaş, char pCinsiyet, uint pMüdürOlduğuBölümKodu) {
        Yaş=pYaş; Cinsiyet=pCinsiyet; MüdürOlduğuBölümKodu=pMüdürOlduğuBölümKodu;
    }
    //Özellikler:
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public uint BölümKodu { get; set; }
    // Yöntemler:
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}
public void DersProgramıHazırla()
{
    Console.WriteLine($"{BölümKodu} Bölümünde Ders Programı Hazırlıyorum...");
}
}

```

Yukarıdaki örnek programdaki sınıfların her birinde **Yaş** ve **Cinsiyet** gibi ortak özellikler (**property**) veya her sınıfta aynı kodu içerecek şekilde **YaşınıSöyle()** ve **CinsiyetiniSöyle()** gibi ortak davranışlar (**behavior**) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanmıştır.

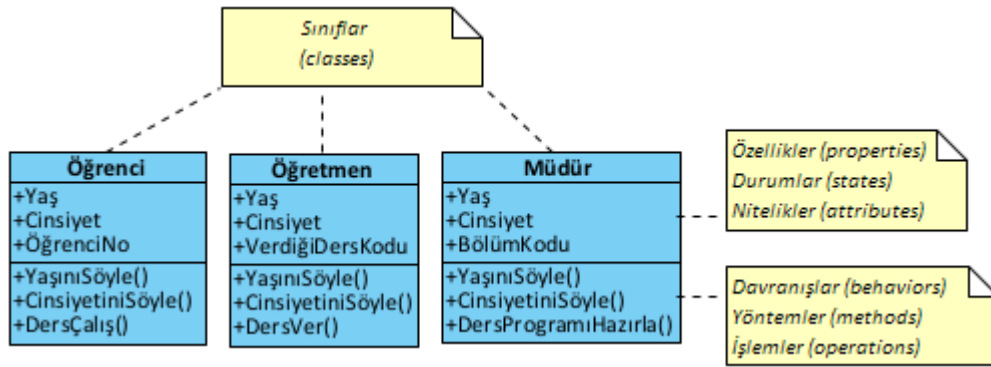
Yazılım analizi yapılırken, takım içerisinde iletişimi kolaylaştırmak için aşağıdaki **sınıf diyagramları** (**class diagram**) kullanılır. Bu diyagramlar UML (**Unified Modeling Language**) dilinin bir parçasıdır. Sonraki bölümde anlatılacaktır. Mevcut okul projesinin UML diyagramı Şekil 8.2’de verilmiştir.

Nesne yönelimli programlamada, örnekteki gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanarak bu sınıflar ortak olmayan özellik ve davranışları barındıracak şekilde yeniden tanımlanabilir. Buna **miras alma** (**inheritance**) veya kısaca **kalıtım** adı verilir. Böylece proje zamanı kısalar ve daha yönetilebilir bir kod elde edilir. Bu durumda miras almış halleriyle okul projesi aşağıdaki gibi kodlanabilir;

```

//ANA PROGRAM:
Öğrenci ali = new Öğrenci(20, 'E', 12000);
ali.YaşınıSöyle(); //Yaşım:20
ali.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK

```



Şekil 8.2: Okul Projesi Sınıfları UML Diyagramı

```
ali.DersÇalış(); //Dersimi Çalışıyorum...
```

```
Öğretmen ilhan = new Öğretmen(50, 'E', 101);
ilhan.YaşınıSöyle(); //Yaşım:50
ilhan.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK
ilhan.DersVer(); //101 Dersini Veriyorum...
```

```
Müdür damla = new Müdür(30, 'K', 5000);
damla.YaşınıSöyle(); // Yaşım:30
damla.CinsiyetiniSöyle(); //Cinsiyetim: KADIN
damla.DersProgramıHazırla(); //5000 Bölümünde Ders Programı Hazırlıyorum...
```

```
//SINIFLAR:
```

```
class Kişi //Taban(base)/Ebeveyn(parent)/Genel(general) Sınıf
{
    //Yapıcı
    public Kişi (uint pYaş, char pCinsiyet)
    {
        Yaş=pYaş; Cinsiyet=pCinsiyet;
    }
    //Ortak Özellikler
    public uint Yaş { get; set; } //sanki "yaş" alan değişkeni varmış gibi kısa özellik tanımı!
    public char Cinsiyet { get; set; }

    // Ortak Davranışlar
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}

class Öğrenci: Kişi //Türemiş(derived)/Çocuk(child)/Özel(special) Sınıf
{
}
```

```

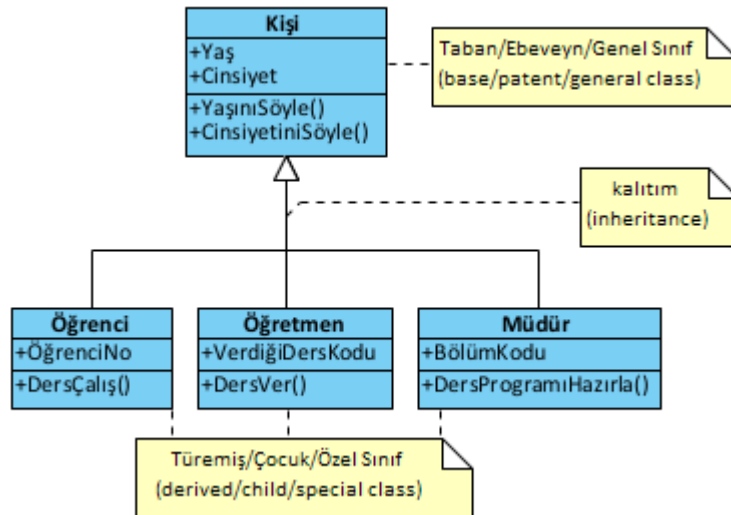
//Yapıcı
public Öğrenci(uint pYaş, char pCinsiyet, uint pOkulNumarası): base(pYaş,pCinsiyet) //base
→ ile taban sınıfın yapıcısı çağrılır
{
    OkulNumarası=pOkulNumarası;
}
//Özellikler
public uint OkulNumarası { get; set; }
//Davranışlar
public void DersÇalış()
{
    Console.WriteLine("Dersimi Çalışıyorum...");
}
}
class Öğretmen: Kişi //Türemiş(derived)/Çocuk(child)/Özel(special) Sınıf
{
    public Öğretmen(uint pYaş, char pCinsiyet, uint pVerdiğiDersKodu): base(pYaş,pCinsiyet)
    → //base ile taban sınıfın yapıcısı çağrılır
    {
        VerdiğiDersKodu=pVerdiğiDersKodu;
    }
    //Özellikler
    public uint VerdiğiDersKodu { get; set; }
    // Yöntemler
    public void DersVer()
    {
        Console.WriteLine($"{VerdiğiDersKodu} Dersini Veriyorum...");
    }
}
class Müdür:Kişi //Türemiş(derived)/Çocuk(child)/Özel(special) Sınıf
{
    public Müdür(uint pYaş, char pCinsiyet, uint pBölümKodu): base(pYaş,pCinsiyet) //base ile
    → taban sınıfın yapıcısı çağrılır
    {
        BölümKodu=pBölümKodu;
    }
    //Özellikler
    public uint BölümKodu { get; set; }
    // Yöntemler
    public void DersProgramıHazırla()
    {
        Console.WriteLine($"{BölümKodu} Bölümünde Ders Programı Hazırlıyorum...");
    }
}

```

Yukarıdaki kod incelendiğinde şunları söyleyebiliriz;

- ◇ Nesne oluşturan ve kullanan ana programda hiçbir değişiklik olmamıştır.
- ◇ Ortak özellik ve davranışları barındıran **Kişi** sınıfı tanımlanmıştır.
- ◇ **Öğrenci**, **Öğretmen** ve **Müdür** sınıflarında ortak özellik ve davranışlar kaldırılmıştır. Bunun yerine her bir sınıf kimliğinin arkasına **:Kişi** kodu eklenerek **Kişi** sınıfından ortak özellik ve davranışlar **miras alınmıştır**.
- ◇ Her bir sınıfın yapıcısına, miras alınan sınıfın özelliklerine ilk değer vermek amacıyla **base** anahtar kelimesi ile yapıcı argümanları geçirilmiştir.
- ◇ **Kişi** olarak tanımlanan ortak özellik ve davranışlar sınıfı, **genel sınıf (general class)**, **taban sınıf (base class)** veya **ebeveyn sınıf (parent class)** olarak adlandırılır.
- ◇ **Öğrenci**, **Öğretmen** ve **Müdür** sınıfları ise **Kişi** taban sınıfından **türemiştir (derived)**. Bu sınıflar genel sınıf olan **Kişi** sınıfının yanında kendine has özel özellik ve davranışlara da sahiptir. Bu nedenle **özel sınıf (private class)**, **türemiş sınıf (derived class)** veya **çocuk sınıf (child class)** olarak adlandırılır. Bu sınıflar mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

Kalıtım uygulanmış haliyle okul projesinin UML diyagramı Şekil 8.3’de verilmiştir;



Şekil 8.3: Kalıtım Uygulanmış Okul Projesi Sınıfları UML Diyagramı

Örneğimizden gidecek olursak, **Kişi** sınıfında yapılacak bir özellik veya davranış eklemesi anında bu sınıftan türetilmiş sınıflara da yansır. Türetilmiş olan **Öğrenci**, **Öğretmen** ve **Müdür** sınıflarında bununla ilgili değişiklik gerekmez.

Değişim yönetimini kolayca yapma üstünlüğünü veren miras almanın (**inheritance**) iki önemli özelliği vardır;

1. Paylaşılan bir ortak kod (**shared code**) vardır.
2. Paylaşılan kodda yapılan değişiklik anında alt sınıflara yansır (**snap change**).

§8.10.1 Miras Alınamayanlar

Taban sınıftan **teknik olarak miras alınamayan** aşağıdaki yapılar vardır;

- ◊ **Yapıcılar (constructor)**: Taban sınıfın yapıcıları türetilmiş sınıfa geçmez. Ancak, türetilmiş sınıfın yapıcısı çalıştığında taban sınıfın yapıcısını (**base** ile) gizli veya açık bir şekilde çağırır.
- ◊ **Sonlandırıcılar (finalizer)**: C# dilinde sonlandırıcılar, miras alınmaz. Her sınıfın kendi yıkım süreci vardır. sonlandırıcılar miras alınmaz ancak hiyerarşik olarak tetiklenirler. Bir nesne yok edilirken önce en alt sınıfın (**derived**), sonra sırasıyla üst sınıfların (**base**) sonlandırıcıları çalışır.

Finalizer Order: DerivedClass → BaseClass → System.Object

Modern C# geliştirmede, sonlandırıcılar tek başına yetersiz görülür çünkü zamanlaması belirsizdir. Bu yüzden **IDisposable** arayüzü ve **Dispose()** metodu ile birlikte kullanılırlar. 8.17 başlığında anlatılacaktır.

- ◊ **Kopya Yapıcılar**: C# dilinde, C++ dilindeki gibi yerleşik bir kopya yapıcı (**copy constructor**) kavramı yoktur. Eğer kendiniz yazarsanız, bu da bir yapıcı olduğu için miras alınmaz.

§8.10.2 protected Erişim Değiştiricisi

Miras alma (Inheritance) konusunu anlatırken karşımıza çıkan en kritik konulardan biri de **protected** erişim değiştiricisinin (**access modifier**) kullanılmasıdır. Bu değiştirici kısaca şöyle tanımlayabiliriz: "Dış dünyaya kapalı (**private**), ama mirasçılara açık."

Bir üyeyi **protected** olarak tanımladığınızda:

- ◊ Sınıfın dışından (programın geri kalanından) bu üyeye erişilemez.
- ◊ Bu sınıftan türetilen (miras alan) tüm sınıflar bu üyeyi sanki kendi içindeymiş gibi kullanabilir.

Bunu anlamak için aşağıdaki örnek verilebilir;

```

Yoneticici mudur = new Yoneticici();
mudur.MaasArtir();
// mudur.Maas = 50000; // HATA: 'Maas' protected olduğu için Main metodundan (dışarıdan)
→ erişilemez.
  
```

```

class Personel
{
  
```



```

private string _tcKimlikNo = "12345678901"; // private: Sadece bu sınıfa özel. Kimse göremez.
protected double Maas = 20000; // protected: Miras alan sınıflar (Yönetici gibi) görebilir.
public void BilgiVer()
{
    Console.WriteLine("Ben bir personelim.");
}
}
class Yoneticici : Personel
{
    public void MaasArtir()
    {
        Maas += 5000; // ERIŞEBİLİR: Maas, üst sınıfta 'protected' olduğu için miras alındı.
        Console.WriteLine($"Yeni Maaş: {Maas}");
        // Console.WriteLine(_tcKimlikNo); // HATA: taban sınıfta _tcKimlikNo 'private' olduğu
        // için türetilmiş sınıftan dahi erişilemez.
    }
}

```

Neden `protected` Kullanırsınız?

- ♦ **Güvenli kapsülleme (encapsulation):** Veriyi tamamen dışarıya açmak (`public`) yerine, sadece o veriyi işleme yetkisi olan "aile üyelerine" (mirasçılara) açmış olursunuz.
- ♦ **Kod Tekrarını Önleme:** Ortak özellikleri üst sınıfta `protected` tutarak, alt sınıfların bu özelliklere doğrudan erişmesini ve yeniden tanımlama yapmamasını sağlarsınız.
- ♦ **Hiyerarşik Kontrol:** Sınıf hiyerarşisi içinde bir veri akışı kurmanıza olanak tanır.

Ayrıca; Eğer alt sınıfların bir özelliğin değerini değiştirmesini (`set`) istiyor ama dış dünyadan bu özelliğin sadece okunmasını (`get`) istiyorsanız, erişim değiştiricilerini hibrit kullanabiliriz. Buna örnek olarak şu özellik tanımını verebiliriz: `public string Ad { get; protected set; }`

§8.10.3 Gölgeleme

Miras aldığınız bir sınıfın içinde, üst sınıfta (base class) zaten var olan bir metodun veya alan değişkeninin ismini tekrar tanımlamak istediğinizde C# derleyicisi bir uyarı verir. "Ben bu kimliği tekrar kullanıyorum ve üst sınıftakini gizlemek istiyorum" demek için `new` belirleyicisini (`new specifier`) kullanırsınız. Bu işleme **üye gizleme (member hiding)** veya gölgeleme adı verilir.

```

Taban taban = new Taban(10);
Türemiş türemiş = new Türemiş(30);
taban.Yöntem(); // Taban Sınıfta 10
türemiş.Yöntem(); /* Türemiş Sınıfta 0: Çünkü; türemiş sınıfta tanımlanan alan değişkeni
→ başlangıç değeri sıfırdır ve taban sınıftaki x gizlenmiştir. */

```

```

public class Taban
{
    public Taban(int x)
    {
        this.x = x;
    }
    public int x;
    public void Yöntem()
    {
        Console.WriteLine($"Taban Sınıfta {x}");
    }
}
public class Türemiş : Taban
{
    public Türemiş(int x) : base(x) { }
    new public int x; //taban sınıftaki kimlikle yeni x değişkeni
    new public void Yöntem() //taban sınıftaki kimlikle yeni Yöntem
    {
        Console.WriteLine($"Türemiş Sınıfta {x}");
    }
}

```

Türemiş sınıfta kullanılan **new** belirleyicisi üyelerin taban sınıfta olan bağı kopartılmıştır. Program çalışırken neye bakacağını referans tipine göre karar verir.

8.11 Geçersiz Kılma

Geçersiz kılma (overriding), nesne yönelimli programlamanın, taban sınıfta önceden tanımlanmış bir yöntemi türemiş bir sınıfta yeniden tanımlamasına olanak tanıyan bir kavramdır. Yani taban sınıfın davranışının türemiş sınıfta değiştirilmesidir.

Geçersiz kılınacak yöntem, taban sınıfta ya **sanal (virtual)** ya da **soyut (abstract)** olarak tanımlanmalıdır. Soyut kavramı 8.6 başlığında verilmişti. Soyut yöntemler **abstract** sıfatıyla tanımlanır ve türetilmiş sınıfta **mutlaka geçersiz kılınmalıdır (override)**. Sanal yöntemler ise **virtual** sıfatıyla tanımlanır ve geçersiz kılınma zorunluğu yoktur. Geçersiz kılınarak yeniden tanımlanan yöntem **override** sıfatı ile tanımlanır. Her iki sınıf ta da yöntemin kimliği aynıdır. Taban sınıftan imal edilen nesne taban sınıfın yöntemini, türemiş sınıftan imal edilen nesne ise türemiş sınıfın yöntemini icra eder.

```
Baba baba = new Baba();
Çocuk çocuk = new Çocuk();
baba.SoyutYap(); //Baba soyutu somutlaştırdı.
baba.Yap(); //Baba bu şekilde yapar ...
çocuk.SoyutYap(); //Çocuk babanın somutlaştırdığını değiştirdi.
çocuk.Yap(); //Çocuk babanı yaptığını başka şekilde yapar...
```

```
//Sınıflar:
abstract class Kişi //Soyut Sınıf
{
    public abstract void SoyutYap(); //Soyut Bir Davranış Tanımı:
}
class Baba: Kişi // Somut Sınıf
{
    public override void SoyutYap() //Soyut Yöntem Mutlaka Geçersiz Kılınmalıdır!
    {
        Console.WriteLine("Baba soyutu somutlaştırdı.");
    }
    public virtual void Yap() // Sanal bir davranış Tanımı:
    {
        Console.WriteLine("Baba bu şekilde yapar ...");
    }
}
class Çocuk : Baba
{
    public override void SoyutYap() //Babanın somutlaştırdığı yöntem de geçersiz kılınabilir
    {
        Console.WriteLine("Çocuk babanın somutlaştırdığını değiştirdi.");
    }
    public override void Yap() // Babanın sanal Davranışı geçersiz kılınıyor. İstenirse
    ↪ yapılmayabilir!
    {
        Console.WriteLine("Çocuk babanı yaptığını başka şekilde yapar...");
    }
}
```

En çok karıştırılan nokta, gölgeleme (**new**) ile geçersiz kılma (**override**) arasındaki farktır. Tablo 8.2 ile konuyu netleştirebilirsiniz:

8.12 Statik Üyeler ve Statik Sınıf

Programda çalışmaya başladığı anda bazı değişkenler ve yöntemlere henüz bellekte tahsis edilmemiş veya istenilen nesneler henüz imal edilmemiştir. Program daha çalışmadan bazı değişken, yöntem ve nesnelerin bellekte hazır olması istenir. Uygulamamızın daha belleğe yüklenirken ve yaşamı boyunca bellekte bulunacak değişken, yöntem ve nesneler **statik (static)** olarak tanımlanır.

Bir sınıfın bir veya birkaç üyesi statik olabilir. Statik olan değişken veya yöntemler yani statik üyeler

Özellik	new (Gölgeleme)	override (Geçersiz Kılma)
Bağlantı Referans Tipi	Üst sınıf ile bağı koparır. Referans tipine göre yöntem çağrılır.	Üst sınıfın davranışını günceller. Nesne tipine göre yöntem çağrılır.
Gereksinim	Üst sınıftaki yöntemin virtual olmasına gerek yoktur.	Üst sınıf yöntem mutlaka virtual veya abstract olmalıdır.

Tablo 8.2: new ile override Karşılaştırması

(**static member**), örnek üyeleri (**instance member**) gibi nesne ile imal edilip nesne ile ölmezler. Bu üyeler program yüklenirken ver bellekte (**data segment**) hazır olurlar. Bu nesnelere statik üyeler, örneğe ait değil sınıfa ait olarak nitelendirilir. Yani program bellekte kaldığı sürece statik sınıf ve üyeleri hayattadır.

Programlarda sıkça kodladığımız programın başladığı **static void Main(string[] args)** yöntemi de bu nedenle **static anahtar kelimesiyle** (**static keyword**) tanımlanır.

Sadece statik üyeleri barındıran **statik sınıflar** (**static class**) da tanımlayabiliriz. Bu sınıflara **yardımcı sınıf** (**utility class**) adı da verilir;

- ◇ Doğası gereği nesne yapıcısı barındırmazlar!
- ◇ Bu sınıflar türetilemez!
- ◇ Sınıfın tüm üyeleri statik olmak zorundadır!

```
string bağlantı = Yardımcı.GetConnection();
Yardımcı.GenelSayaç++;
ulong f = Yardımcı.Faktöriyel(5);
```

```
static class Yardımcı /* Statik sınıf */
{
    public static ulong GenelSayaç = 0;
    public static string GetConnection()
    {
        return "Data Source=(LocalDB)\MSSQLLocalDB; Integrated Security=True;Connect
        ↳ Timeout=30;Encrypt=True";
    }
    public static ulong Faktöriyel(uint n)
    {
        if (n > 1) return n * Faktöriyel(n - 1);
        else return 1;
    }
}
```

8.13 İşleçlerin Aşırı Yüklenmesi

İşleçleri aşırı yüklenmesi (**operator overloading**), dilin yerleşik operatörlerine (+, -, *, == gibi) kendi oluşturduğunuz sınıflar için özel anlamlar yüklemenizi sağlar. Aşırı yüklenecek işleçlere ilişkin yöntemler;

- ◇ **Statik Olma Zorunluluğu:** Tüm operatör metotları **public static** olmak zorundadır.
- ◇ **Parametre Sayısı:** Tek işlenenle işlem yapan işleçler (**unary operator**) bir, iki işlenen ile işlem yapan ikili işleçler (**binary operator**) iki parametre alır.
- ◇ **En Az Bir Nesne:** Parametrelerden en az biri, işlecin tanımlandığı sınıf tipinde olmalıdır.

```
public class Koordinat
{
    public int X { get; set; }
    public int Y { get; set; }

    // + Operatörünü Vektor sınıfı için tanımlıyoruz:
    public static Koordinat operator +(Koordinat v1, Koordinat v2)
    {
        return new Vektor { X = v1.X + v2.X, Y = v1.Y + v2.Y };
    }
}
```

C# dilinde her işleci aşırı yükleyemezsiniz. Aşağıda aşırı yüklenebilecek işleç listesi tablo olarak verilmiştir;

Kategori	İşleçler
Tekli (Unary)	<code>+, -, !, ~, ++, --, true, false</code> (<i>true ve false çift olarak yüklenmelidir</i>)
İkili (Binary)	<code>+, -, *, /, %, &, , ^, <<, >></code>
Karşılaştırma	<code>==, !=, <, >, <=, >=</code> (<i>çiftler olarak yüklenmelidir</i>)
Yüklenemez	<code>&&, , =, ?:, +=, -=, *=, /=, nokta (.), new</code>

Tablo 8.3: Aşırı Yüklenebilecek İşleçler

İşleçleri ön yüklerken aşağıdakilere dikkat etmek gerekir;

- ◊ **Eşli Yükleme Zorunluluğu:** Eğer `==` işlecini yüklüyorsanız, mutlaka `!=` işlecini de yüklemek zorundasınız. Aynı kural `<` ve `>`, `<=` ve `>=`, `true` ve `false` çiftleri için de geçerlidir.
- ◊ **Mantıksal İşleçler:** `&&` ve `||` işleçler doğrudan yüklenemez; ancak `&`, `|`, `true` ve `false` işleçlerini yükleyerek bu davranış dolaylı yoldan kontrol edilebilir.
- ◊ **Atama İşleçleri:** `+=` işleci doğrudan yüklenemez. Ancak siz `+` işleci yüklerseniz, C# otomatik olarak `+=` işlemini bu yeni kurala göre gerçekleştirir.

8.14 Sabit Alan Değişkenleri ve Yalnız Okunabilir Nesneler

Sınıflardaki alan değişkenlerinin (**field**) nesnelere ilişkin durumları (state) yani verileri tuttuğu anlatılmıştı. Sabitler, derleme zamanında bilinen ve ömrü boyunca değişmeyen değişmez değerlerdir. Sabit alan değişkenleri ve yerel değişkenler `const` sıfatıyla tanımlanırlar.

Metinler (**string**) dışındaki referans tiplerine ilişkin sabitlere yalnızca `null` olarak ilk değer verilebilir! Sınıflar (**class**), yapılar (**struct**) ve diziler (**array**) dahil olmak üzere kullanıcı tanımlı referans tipler `const` sıfatıyla tanımlanamaz! Çalışma zamanında yapıcı ile bir kez imal edilen ve bundan sonra değiştirilemeyen bir nesne, yapı veya dizi oluşturmak için `readonly` sıfatı kullanılır.

```
const float PI = 3.1415f;
Takvim t = new Takvim();
t.Yaz();
/*
Ay Sayısı:12
Saat Sayısı:8786 */
Console.WriteLine(Takvim.TakvimAdı);
/*
const alanlar static olarak algılanır ve Sınıf İsmi ile erişilir.
*/
Console.WriteLine(t.günler[0]); //Pazartesi
/*
readonly alanlar, static olarak algılanmaz ve nesne üzerinden erişilir.
*/

class Takvim
{
    public const int AySayısı = 12;
    public const int GünSayısı = 365;
    public const int HaftaSayısı = 52;
    public const int SaatSayısı = GünSayısı * 24 + 6;
    public const string TakvimAdı = "Takvim";
    public readonly string[] günler = { "Pazartesi", "Salı", "Çarşamba", "Perşembe", "Cuma",
        ↪ "Cumartesi", "Pazar" };
    public void Yaz()
    {
        Console.WriteLine($"Ay Sayısı:{AySayısı}");
        Console.WriteLine($"Saat Sayısı:{SaatSayısı}");
    }
}
```

`readonly` anahtar kelimesi (**readonly keyword**) aşağıdaki bağlamlarda kullanılabilen bir sıfattır.

1. **Salt okunur bir alan değişkeni tanımlanmasında:** Alana (**field**) atama yalnızca bildirimde veya aynı sınıftaki bir yapıcıda gerçekleşir.

```
class Takvim
{
```

- ```

 public readonly int AySayısı=12;
 public readonly int HaftaSayısı;
 public Takvim()
 {
 HaftaSayısı = 365 / 7;
 /* Sınıf içinde bile olsa Başka yerde bu alanlar değiştirilemez. */
 }
}

```
2. Salt okunur bir yapı tanımında; Yapı (**struct**) tipinin değiştirilemez olduğunu belirtir.
- ```

var p1 = new Koordinat(0, 0);
Console.WriteLine(p1); // (0, 0)
var p2 = p1 with { X = 3 };
Console.WriteLine(p2); // (3, 0)
var p3 = p1 with { X = 1, Y = 4 };
Console.WriteLine(p3); // (1, 4)

```
- ```

public readonly struct Koordinat
{
 public Koordinat(double x, double y)
 {
 X = x;
 Y = y;
 }
 public double X { get; init; }
 public double Y { get; init; }
 public override string ToString()
 {
 return $"({X}, {Y})";
 }
}

```
3. Örnekleme yapılarak: Nesne imal edilerek (**instantiation**) salt okunur bir üye nesne tanımlanıyorsa; bu örnek üyesinin durumunu değiştirmedigini belirtir.
- ```

Kişi ali = new Kişi("Ali", "YILMAZ");
ali.Yaz();

```
- ```

class Kitap
{
 public string Adı { get; set; }
 public Kitap(string adı)
 {
 this.Adı = adı;
 }
}
class Kişi
{
 public string Adı { get; set; }
 public string Soyadı { get; set; }
 public readonly Kitap matematikKitabı = new Kitap("Calculus I");
 public readonly Kitap kitap;
 public Kişi(string adı, string soyadı)
 {
 this.Adı = adı;
 this.Soyadı = soyadı;
 kitap = new Kitap("C# Programlama");
 }
 public void Yaz()
 {
 Console.WriteLine($"Adı:{Adı}, Soyadı:{Soyadı}");
 Console.WriteLine($"Matematik: {matematikKitabı.Adı}");
 Console.WriteLine($"Kitap: {kitap.Adı}");
 }
}

```

4. Bir `ref readonly` yöntemi dönüşünde: `readonly` kelimesi yöntemin bir referans döndürdüğünü ve bu referansa yazmalara izin verilmediğini belirtir.

## 8.15 Değiştirilemez Nesneler

Nesne imal edildikten sonra, **durumları değiştirilebilir** (**mutable**) nesnelerdir. Bir nesnenin durumlarının değiştirilebilir olması, dahili verilerinin değiştirilebileceği ve bu da nesnenin bütünlüğünün korunmasında olası yan etkilere ve zorluklara yol açabileceği anlamına gelir. Değiştirilebilir sınıfların yaygın örnekleri arasında, elemanlarının eklenebildiği, kaldırılabilirdiği veya değiştirilebildiği birçok koleksiyon türü bulunur.

Değiştirilebilir nesneler yerine değiştirilemez nesneler kullanmak istediğimiz durumlar vardır. Değiştirilemez nesneler **iş parçacığı güvenlidir** (**thread-safe**), ancak diğer yandan birçok durumda performans sorunlarına neden olabilirler. Her geliştiricinin günlük olarak kullandığı en popüler değiştirilemez nesne metin değişkenleridir (**string**). Metinlerle yapılan her işlem sonrasında yeni bir metin ortaya çıkar. Değiştirilemez bir nesne elde etmek için sınıflardaki alanlar `readonly` tanımlanır. Bu sınıfın yapıcısında ilk değerler verilir ve nesne imalatı olduktan sonra bu alanlar değiştirilemez.

```
Takvim t = new Takvim();
t.Yaz();
Console.WriteLine($"Saat Sayısı:{t.SaatSayısı}");
Console.WriteLine(t.TakvimAdı.ToUpper());
/* TakvimAdı immutable string sınıfındandır. Burada mevcut değeri "Takvim" aynen kalır ve büyük
→ harfe çevrilmiş olan yeni bir string imal edilir. */
```

```
class Takvim
{
 public readonly int AySayısı;
 public readonly int GünSayısı;
 public readonly int HaftaSayısı;
 public readonly int SaatSayısı;
 public readonly string TakvimAdı;
 public Takvim()
 {
 AySayısı = 12;
 GünSayısı = 365;
 HaftaSayısı = GünSayısı / 7;
 SaatSayısı = GünSayısı * 24 + 6;
 TakvimAdı = "Takvim";
 }
 public void Yaz()
 {
 Console.WriteLine($"Hafta Sayısı:{HaftaSayısı}");
 Console.WriteLine($"Gün Sayısı:{GünSayısı}");
 }
}
```

## 8.16 Arayüzler

**Arayüzler** (**interface**) nesnelerin sahip oldukları **rollerdir** (**role**). Ancak saf nesne yönelimli dillerde arayüzler **interface** saklı kelimesiyle tanımlanırlar.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav veya değerlendirme yapma, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Arayüzler de **referans tiplerdir**. Bir sınıfa, bir veya **birden fazla arayüz uygulanabilir** (**interface implementation**). Bir sınıfın bir arayüzü miras almasına **arayüz gerçekleştirme** (**interface realization**) adı da verilir. Arayüzler içinde yöntem imzaları (**signature**) yazılır. Yani sadece yöntem başlığı yazılır. Gövdesi yazılmaz. Ayrıca yöntemler için **erişim belirleyici** (**access modifier**) kullanılmaz.

```
Kişi ilhan = new Kişi("İlhan", "ÖZKAN"); //imal edilen nesne tüm arayüzdeki davranışları
→ gösterebilir.
ilhan.AdınıSöyle(); // İlhan ÖZKAN
ilhan.Öğret(); // Öğretiyorum...
```

```
ilhan.GitarÇal(); // Gitar Çalıyorum...
ilhan.HanımaYardımEt(); // Hanıma Yardım Ediyorum...
```

```
IBaba baba = ilhan; //imal edilen nesne sadece bir roldeki davranışları da gösterebilir:
baba.ÇocuğaBak(); // Çocuğa Bakıyorum...
baba.HanımaYardımEt(); // Hanıma Yardım Ediyorum...
İMüzişyen müzişyen = ilhan;
müzişyen.GitarÇal(); // Gitar Çalıyorum...
müzişyen.PiyanoÇal(); // Piyano Çalıyorum...
İÖğretmen öğretmen = ilhan;
öğretmen.DersNotuHazırla(); // Ders notu hazırlıyorum...
öğretmen.Öğret(); // Öğretiyorum...
```

```
interface IBaba //Baba Rolündeki Davranışlar:
{
 void ÇocuğaBak(); // Aarayüzde yöntemin gövdesi olmaz!
 void HanımaYardımEt();
}
interface İMüzişyen //Müzişyen Rolündeki Davranışlar:
{
 void GitarÇal();
 void PiyanoÇal();
}
interface İÖğretmen//Öğretmen Davranışları:
{
 void Öğret();
 void DersNotuHazırla();
}
class Kişi : IBaba, İMüzişyen, İÖğretmen //Arayüz Gerçekleştirme!
{
 /*
 Arayüz Gerçekleştirme/Uygulama(interface realization/implementation):
 Gerçekleştirilen IBaba, İMuzişyen ve İOğretmen arayüzlerinde bulunan yöntemler
 Kişi sınıfında hepsi gövdeleriyle tanımlanmalıdır!
 */
 public string Adı { get; set; }
 public string Soyadı { get; set; }
 public Kişi(string adı, string soyadı)
 {
 this.Adı = adı;
 this.Soyadı = soyadı;
 }
 public void AdınıSöyle()
 {
 Console.WriteLine($"{Adı} {Soyadı}");
 }
 public void ÇocuğaBak()
 {
 Console.WriteLine("Çocuğa Bakıyorum..");
 }
 public void HanımaYardımEt()
 {
 Console.WriteLine("Hanıma Yardım Ediyorum..");
 }
 public void PiyanoÇal()
 {
 Console.WriteLine("Piyano Çalıyorum..");
 }
 public void GitarÇal()
 {
 Console.WriteLine("Gitar Çalıyorum..");
 }
}
```



```

}
public void Öğret()
{
 Console.WriteLine("Öğretiyorum..");
}
public void DersNotuHazırla()
{
 Console.WriteLine("Desr Notu Hazırlıyorum..");
}
}

```

## 8.17 Sonlandırıcılar

**Sonlandırıcılar** (**finalizers**), bir nesnenin yaşam döngüsünün "gün batımı" aşamasını temsil eder. C++ dilindeki yıkıcılara (**destructors**) benzer bir söz dizimine sahip olsalar da, C# dilinin bellek yönetim sistemi olan **çöp toplama** (GC) nedeniyle çalışma mantıkları oldukça farklıdır.

Sonlandırıcı, bir sınıfın örneği (**instance**) **bellekten temizlenmeden hemen önce** çöp toplayıcı tarafından otomatik olarak çağrılan özel bir yöntemdir. Temel amacı, nesne yok edilmeden önce yönetilmeyen kaynakları (**unmanaged resources**) serbest bırakmaktır.

Yazım Kuralları:

- ◊ Sınıf kimliğinin başına eğik çizgi (~) karakteri konularak tanımlanır.
- ◊ **Parametre almazlar ve geri dönüş tipleri yoktur.**
- ◊ Erişim belirleyicisi (**public**, **private** vb.) **sıfatları olmaz.**
- ◊ Bir sınıfta **sadece bir tane** sonlandırıcı bulunabilir.

```

class DosyaYonetici
{
 ~DosyaYonetici() // Sonlandırıcı (Finalizer)
 {
 // Nesne bellekten silinirken yapılacak işlemler
 Console.WriteLine("Nesne imha ediliyor, kaynaklar serbest bırakıldı.");
 }
}

```

C# öğrenenlerin en çok düştüğü hata, sonlandırıcının nesne kapsam dışına çıktığı an çalışacağını sanmaktır;

- ◊ **Belirsiz Zamanlama** (**non-deterministic**): C++ dilinin aksine, C# dilinde sonlandırıcının ne zaman çalışacağını bilemezsiniz. **Çöp toplayıcı** (GC), ne zaman uygun görürse o zaman çalışır.
- ◊ **Yönetilmeyen Kaynaklar**: Sonlandırıcılar, açık dosyalar, ağ bağlantıları veya donanım portları gibi .NET dışı kaynakları kapatmak için kullanılır. Normal bir değişken (**int**, **string** vb.) için sonlandırıcı yazmaya gerek yoktur; bunları çöp toplayıcı zaten temizler.

Modern C# geliştirmede, sonlandırıcılar tek başına yetersiz görülür çünkü zamanlaması belirsizdir. Bu yüzden **IDisposable** arayüzü ve **Dispose()** yöntemi ile birlikte kullanılırlar. Buna **kullan-at deseni** (**disposable pattern**) adı verilir.

```

Console.WriteLine("--- İşlem Başlıyor ---");
using (KaynakYonetici yonetici = new KaynakYonetici()) // using bloğu dışına çıkıldığı an
 → Dispose() otomatik tetiklenir.
{
 Console.WriteLine("Kaynaklar kullanılıyor...");
 // Burada dosyalarla veya ağ ile ilgili işlemler yapılır.
}
Console.WriteLine("--- İşlem Bitti ve Kaynaklar Serbest Bırakıldı ---");

```

```

class KaynakYonetici : IDisposable
{
 private bool _disposed = false;
 // Kullanıcının manuel çağırabileceği temizlik metodu
 public void Dispose()
 {
 Console.WriteLine("Dispose() Yöntemi Çalıştı.");
 }
}

```

```

 Temizle(true);
 GC.SuppressFinalize(this); // Çöp toplayıcı (GC)'ye "sonlandırıcıyı çalıştırmaya gerek
 ↳ kalmadı" der.
 }
 ~KaynakYonetici() {
 Temizle(false);
 }
 protected virtual void Temizle(bool manuelTemizlik)
 {
 if (_disposed) return;
 if (manuelTemizlik)
 {
 /* Yönetilen kaynakları temizle */
 }
 // Yönetilmeyen kaynakları her durumda temizle
 _disposed = true;
 }
}

```

## 8.18 Genişleme Yöntemleri

**Genişleme yöntemleri** (**extension method**), C#3.0+ ile hayatımıza girmiştir. Genişletme yöntemleri, yeni bir türetilmiş tip oluşturmadan, yeniden derlemeden veya orijinal türü başka şekilde değiştirmeden mevcut türlere davranışı genişletir ve ekler. Özellikle geliştirmek istediğiniz bir tipin kaynağını değiştiremediğinizde faydalıdırlar. Genişletme yöntemleri **int**, **char**, **string** gibi sistem tipleri, üçüncü taraflarca kütüphanelerde tanımlanan tipler ve **class** gibi kendiniz tanımladığınız tipler için oluşturulabilir. Genişletme yöntemi orijinal tipin bir üye yöntemiymiş gibi çağrılabilir.

Bir genişletme yöntemi statik olmalı ve ilk parametresi her zaman **this** anahtar sözcüğünden önce gelmeli, ardından genişlettiğiniz nesnenin "geçerli" örneğine başvurmak için tanımlayıcı gelmelidir;

```

class Program
{
 static void Main()
 {
 // aşağıdaki atama String.ToUpper() yöntemini çağırır.
 var myString = "Hello World!".ToUpper();
 /*
 Aşağıdaki yöntem Extensions sınıfının StringExtensions.Shorten() yöntemini çağırır
 */
 var newString = myString.Shorten(5);
 // Aşağıda ise genişletilmiş yöntem çağırılmıştır.
 var newString2 = StringExtensions.Shorten(myString, 5);
 }
}

static class StringExtensions
{
 public static string Shorten(this string text, int length)
 {
 return text.Substring(0, length);
 }
}

```

Bir sınıfın kaynak kodu mevcut değilse veya mevcut sınıfta değişiklik yapma izniniz yoksa, genişleme yöntemlerinin mevcut sınıfa yeni yöntemler ekleyerek bir sınıfın işlevselliğini genişletmek için bir yaklaşım olarak kullanılabileceğini söyleyebiliriz. Hatırlamanız gereken en önemli nokta, genişleme yöntemlerinin statik bir sınıfın statik yöntemleri olduğu, ancak genişletilmiş tipteki örnek yöntemlermiş gibi çağrılacaklarıdır.

## 8.19 Sınıf Çeşitleri

### §8.19.1 Soyut Sınıf

**Soyut sınıflar** (**abstract class**) da tanımlanabilir ancak bu sınıflardan nesne imal edilemez. Yani bu sınıfın bir **örneği** (**instance**) **olamaz!** Daha çok **kavramsal olarak kullanılır!** Soyut bir sınıfın en az bir soyut yöntemi (**abstract method**) olur. Soyut sınıflar hangi davranışı göstereceğini söyler ama nasıl davranış göstereceğini

anlatmaz. Bu nedenle **abstract** kelimesiyle tanımlanan **soyut yöntem** (**abstract method**) için gövde tanımlanmaz. Aşağıda soyut bir sınıf tanımı verilmiştir;

```
//Soyut soyutnesne = new Soyut(); //HATA: Soyut sınıftan nesne imal edilemez!

abstract class Soyut
{
 public abstract void Yöntem(); //Soyut Yöntemler için gövde tanımlanmaz!
}
```

### §8.19.2 Somut Sınıf

**Somut sınıflardan** (**concrete class**) nesne imal edilebilir yani örneği (**instance**) olabilir. Bu sınıflarda davranış ve durumlar eksiksiz tanımlıdır. Bir sınıf tanımlarken soyut bir sınıfa genel davranış ve durumları toplamak ve ondan miras alarak somut sınıfları tanımlamak her zaman iyi bir seçimdir.

```
public class Somut : Soyut
/* Somut (concrete) sınıf, her zaman soyut sınıftan miras almak zorunda değildir. */
{
 public override void Yöntem()
 /* soyut yöntem, soyut olan ve miras alınan Yöntem" metodunu * geçersiz kılar (override)
 Ve mutlaka ete kemiğe bürünen bir davranışı tanımlamak zorundadır. */
 {
 /* soyut olarak gösterilecek davranış burada kodlanır. */
 }
}
```

### §8.19.3 Taban Sınıf

**Taban sınıf** (**base class**), miras alınan sınıf veya genel özellik ve davranışların toplandığı yani genelleştirmenin yapıldığı genel sınıf (**general class**) veya ebeveyn (**parent**) sınıftır.

```
public class TabanSınıf
{
 public virtual void Yöntem()
 {
 }
}
```

### §8.19.4 Türemiş Sınıf

**Türemiş sınıf** (**derived class**), miras alan sınıf veya davranış ve durumlar hakkında uzmanlaşmış yani uzman (**special class**) veya çocuk (**child class**) sınıftır. Türemiş ya da miras almış sınıf.

```
public class TüreyenSınıf : TabanSınıf
{
 public override void Yöntem()
 {
 }
}
```

### §8.19.5 Kök Sınıf

**Kök sınıf** (**root class**), babası olmayan ya da yetim sınıf olup aynı zamanda baba sınıftır.

### §8.19.6 Yaprak Sınıf

**Yaprak sınıf** (**leaf class**), kendisinden henüz miras alan bir sınıf olmayan bir sınıftır. Kısaca türememiş sınıftır.

### §8.19.7 Kısır Sınıf

**Kısır sınıf** (**sealed class**) kendisinden türeyen bir sınıf tanımlayamayan bir sınıftır. **Kısır yöntem** (**sealed method**) ise türeyen sınıfta geçersiz kılınamayan (**override**) yöntemdir.

```
public class Sınıf
{
 public virtual sealed void Yöntem()
 { // Kısır bir Yöntem tanımlanıyor
 }
}
```

```

public class Çocuk : Sınıf
{
 public override void Yöntem() /* Derleme Sırasında Hata: Kısır yöntem geçersiz kılınamaz
 → (override edilemez) */
 {
 }
}
public sealed class Kısır //Kısır (sealad) sınıf: miras bırakmaz.
{
 public Kısır() { }
 public void Yöntem()
 {
 }
}
public class Türemiş : Kısır /* Derleme Sırasında Hata: Kısır sınıftan miras alınamaz. */
{
}

```

### §8.19.8 Tekil Sınıf

**Tekil sınıftan (singleton class)** yalnızca bir nesne imal edilebilir. Yani yalnızca bir örneği vardır. Bunun olabilmesi için mahrem (**private**) yapıcısı olması gerekir. Tekil Nesne başlığında incelenecektir.

### §8.19.9 Yardımcı Sınıf

**Yardımcı sınıf (utility class)**, üyeleri statik olan sınıflardır. Ayrıca evrensel yapılandırma ayarlarını (**configuration settings**) veya değişmezleri (**literal**) depolamak için kullanılabilir. Bir kaynak havuzunu (önbellek, veri tabanı bağlantı havuzu, vb.) yönetmek ve örnekler arasında paylaşılan bir günlük sistemi uygulamak için yararlıdır. Bunların dışında, izleme yöntemi (**trace method**) çağrılarını için de kullanılabilir. Yardımcı sınıflar statik (**static**) sınıflardır ve bu sınıflardan nesne imal edilemez.

### §8.19.10 Kısmi Sınıf

**Kısmi sınıf (partial class)**, bir sınıfın farklı yerlerde parçalar halinde tanımlanmasıdır. Kısmi sınıf bir dosya içinde tanımlanabileceği gibi kod tabanında herhangi bir yerde tanımlanabilir.

```

Kişi kişi = new Kişi();
kişi.AdınıSöyle();
kişi.SoyadınıSöyle();
kişi.AdınıTamSöyle();

```

```

partial class Kişi
{
 string Adı { get; set; }
 public void AdınıSöyle()
 {
 Console.WriteLine($"Adım:{Adı}");
 }
}

```

//Aynı dosyanın başka bölümünde ya da projenin bir başka kaynak dosyasında olabilir

```

partial class Kişi
{
 public Kişi()
 {
 Adı = "İlhan";
 Soyadı = "Özkan";
 }
 string Soyadı { get; set; }
 public void SoyadınıSöyle()
 {
 Console.WriteLine($"Soyadı:{Soyadı}");
 }
 public void AdınıTamSöyle()

```

```

{
 Console.WriteLine($"Adı ve Soyadı:{Adı} {Soyadı}");
}
}

```

## 8.20 Nesnelerle İlgili İşleçler

### §8.20.1 Tip Kontrol İşleci

**is** işleci (**is operator**) bir nesnenin hangi veri tipinde (bir sınıfa ait olup olmadığını) kontrol eder. Nesnenin kendisini değiştirmez, sadece "Bu nesne bu veri tipinde mi?" sorusunu sorar.

```

object obj = "Hello, World!";
if (obj is string)
{
 Console.WriteLine("obj nesnesinin veri tipi stringdir."); // Bunu konsola yazar
}

```

Bu işleç, C#7.0+ sonrasında desen seçimi (**pattern matching**) mantığıyla çalışır. Modern C# sürümlerinde **is** operatörü ile hem kontrol yapıp hem de aynı satırda değişken oluşturabilirsiniz.

```

if (nesne is string metin) // hem tip kontrolü hem de metin değişkeni oluşturuldu
{
 // Eğer nesne string ise, 'metin' isminde bir değişken oluşturulur
 Console.WriteLine($"Metnin uzunluğu: {metin.Length}");
}

```

Sınıf hiyerarşisinde bu işlecin nasıl kullanıldığını anlatan örnek aşağıda verilmiştir;

```

public class Sekil { }
public class Kare : Sekil { public void KareCiz() { } }

public void SekilIsle(Sekil s)
{
 if (s is Kare k) // s bir Kare mi? Öyleyse 'k' değişkeniyle kullan.
 {
 k.KareCiz();
 }
}

```

### §8.20.2 Güvenli Tip Dönüşüm İşleci

**Güvenli tip dönüşüm işleci** olan **as** işleci (**as operator**), bir nesneyi belirli veri tipine dönüştürmeye çalışır. Eğer dönüşüm başarısız olursa program hata fırlatmaz, bunun yerine geriye **null** döndürür. Dönüşüm, sadece referans tipleri ve boş olabilir (**nullable**) tipler ile çalışır. **try-catch** bloğuna gerek kalmadan dönüşüm denemesi yapmanızı sağlar.

```

object veri = 123;
string metin = veri as string; // int, string'e dönüşemez.
if (metin == null)
{
 Console.WriteLine("Dönüşüm başarısız, veri bir string değil.");
}

```

Neden **int x = nesne as int;** şeklinde bir kod derleme hatası verir? Çünkü **int** bir değer tipidir (value type) ve asla **null** olamaz. **as** işleci ise başarısızlık durumunda **null** döndürmek üzere tasarlanmıştır. Değer tiplerinde güvenli dönüşüm yapmak istiyorsanız **is** operatörünü kullanmalı veya **Nullable<int>** yani **int?** tercih etmelisiniz.

### §8.20.3 Eşitlik İşleci

C# dilinde iki farklı eşitlik türü vardır. Birincisi referans eşitliği, ikincisi ise değer eşitliğidir. Değer eşitliği, eşitliğin yaygın olarak anlaşılan anlamıdır. İki nesnenin aynı değerleri içermesi anlamına gelir.

```

object a = new object();
object b = a;
System.Object.ReferenceEquals(a, b); //true

```

```

Console.WriteLine((2 + 2) == 4); // değer eşitliği: true
object s = 1;
object t = 1;
Console.WriteLine(s == t); //referans eşitliği, farklı nesneler: false

string a = "hello"; //const
string b = String.Copy(a); //instance
string c = "hello"; //const
Console.WriteLine(a == b); // metin değerlerini karşılaştırma: true

Console.WriteLine((object)a == (object)b); //metin referanslarını karşılaştırma: false
Console.WriteLine((object)a == (object)c); //metinlerin ikisi de sabit: true

```

## 8.21 İsim Uzayları

**İsim uzayları** (**namespaces**), projeyi mantıksal bölümlere ayırmak ve **isim çakışmalarını** (**naming collisions**) önlemek için kullanılan hayati bir yapıdır. Büyük bir kütüphaneyi binlerce kitabın olduğu bir oda yerine, kategorilere ayrılmış raflar olarak düşünün; isim uzayları işte bu raflardır.

### §8.21.1 İsim Uzayı Nedir?

Bir projede aynı isimde iki farklı sınıf (Müşteri, Dosya, vb.) tanımlamanız gerekebilir. Örneğin, hem "Muhasebe" modülünde hem de "Satış" modülünde bir Fatura sınıfı olabilir. İsim uzayları, bu sınıfları birbirinden ayırmanızı sağlar:

```

Şirketim.Muhasebe.Fatura
Şirketim.Satış.Fatura

```

**İsim Uzayı Tanımlama ve Kullanımı** Bir sınıfın hangi isim uzayına ait olduğunu **namespace** anahtar kelimesi ile belirtirsiniz.

```

namespace VeriYönetimi.Modeller
{
 public class Kullanıcı
 {
 public string Ad { get; set; }
 }
}

```

Bu sınıfa başka bir yerden erişmek için iki yolunuz vardır:

1. Tam Adını Kullanmak (Fully Qualified Name)  
VeriYönetimi.Modeller.Kullanıcı k = new VeriYönetimi.Modeller.Kullanıcı();
2. **using** Direktifi ile Dahil Etmek (Önerilen)  
**using VeriYönetimi.Modeller;**  
// Artık doğrudan sınıf adını yazabilirsiniz  
Kullanıcı k = new Kullanıcı();

### §8.21.2 Hiyerarşik İsim Uzayları

İsim uzayları, nokta işleci (**dot operator**) (.) ile hiyerarşik olarak iç içe dallandırılabilir. Bu, projenin büyüklüğüne göre derinleşebilir:

```

İsimUzay1.KullanıcıArayüzü.Kullanıcı.Kullanıcı kullanıcı1 = new
→ İsimUzay1.KullanıcıArayüzü.Kullanıcı.Kullanıcı
{
 Ad = "Ahmet",
 Soyad = "Yılmaz",
 Tip = İsimUzay1.KullanıcıArayüzü.Kullanıcı.KullanıcıTipi.ADMIN
};
kullanıcı1.Ad = "Ali"; // Değişiklik yapıldı

namespace İsimUzay1
{
 namespace KullanıcıArayüzü
 {

```

```

namespace Kullanıcı {
 enum KullanıcıTipi { ADMIN, RAPORTÖR, İZLEYİCİ }
 class Kullanıcı
 {
 public string Ad { get; set; }
 public string Soyad { get; set; }
 public KullanıcıTipi Tip { get; set; }
 }
}

```

### §8.21.3 Dosya Kapsamlı İsim Uzayları

C#10.0+ ile birlikte, dosyanın tamamını kapsayan isim uzayları (**file-scoped namespaces**) geldi. Bu sayede gereksiz süslü parantezlerden `{ }` ve girintilerden kurtulabilirsiniz:

```

// Eski yöntem (Block-scoped)
namespace Projem {
 class A
 {
 }
}

// Yeni yöntem (file-scoped) - Daha temiz kod!
namespace Projem;
class A
{
}

```

### §8.21.4 İsim Uzaylarında Takma İsim

**Alias Kullanımı:** İsimler çok uzunsa veya çakışma varsa **takma isim** (**alias**) verilir.

```

using System.Text;

var sb = new StringBuilder(); //var sb = new System.Text.StringBuilder(); yerine
 using static: Bir sınıfın statik yöntemler sınıf adını yazmadan erişmenizi sağlar.
using static System.Math; //-> Artık Math.PI yerine sadece PI yazabilirsiniz.

```

### §8.21.5 Standart .NET İsim Uzayları

Şu ana kadar üstü örtük olarak birçok isim uzayını kullandık. Bunlar;

- ◊ **System:** Temel veri türleri, konsol işlemleri.
- ◊ **System.Collections.Generic:** Listeler, sözlükler.
- ◊ **System.Linq:** Veri sorgulama araçları.
- ◊ **System.IO:** Dosya okuma ve yazma.

Bir sınıfı herhangi bir **namespace** bloğu içine almazsak o sınıf **küresel isim uzayı** (**global namespace**) içinde yer alır. Küçük projelerde sorun olmasa da, büyük projelerde bu durum "isimlendirme kirliliğine" yol açar ve profesyonel yazılım geliştirmede asla önerilmez.

## 8.22 Düşün ve Çöz

- ◊ **Düşün:** Kapsülleme (**encapsulation**) neden yalnızca alanları **private** yapmaktan ibaret değildir? Özel-lik (**property**) kullanımının **getter/setter** yöntemlerine göre C#'a özgü avantajları nelerdir?
- ◊ **Düşün:** Miras alma (**inheritance**) ile kompozisyon (**composition**) 'is-a' ve 'has-a' ilişkileri üzerinden karşılaştırınız. Tasarım Desenleri literatüründeki 'Kalıtım yerine kompozisyonu tercih et' ilkesini somut bir örnekle açıklayınız.
- ◊ **Düşün:** C#'ta çok biçimlilik (**polymorphism**) **virtual/override** mekanizmasıyla nasıl sağlanır? Erken bağlama (**early binding**) ile geç bağlama (**late binding**) arasındaki farkı bir hayvan sınıfı hiyerarşisi örneğiyle gösteriniz.



- ◇ **Düşün:** Arayüz (**interface**) ile soyut sınıf (**abstract class**) arasındaki fark nedir? C#8 ile arayüzlere varsayılan yöntem gövdesi eklenebilmesi bu ikisi arasındaki sınırı nasıl değiştirmiştir?
- ◇ **Düşün:** Statik üyeler (**static members**) nesne yönelimli tasarımın hangi ilkesiyle çelişebilir? Utility sınıflarını tamamen statik yapmak ne zaman mantıklı, ne zaman sorunludur?
- ◇ **Düşün:** C#'ta sonlandırıcı (**finalizer/destructor**) kullanımı neden önerilmez? **IDisposable** arayüzü ve **using** bloğu, sonlandırıcı yerine neden tercih edilmelidir?
- ◇ **Düşün:** **new** anahtar kelimesi gölgeleme (**shadowing**) amacıyla kullanıldığında çok biçimliliği nasıl kırar? **override** ile **new** arasındaki farkı bir test senaryosunda gözlemleyebileceğiniz bir örnek kurunuz.
- ◇ **Çöz:** Bir kütüphane sistemi modelleyin: Materyal (soyut), Kitap, Dergi ve DVD sınıflarını oluşturun. Her birinin ödünç alınabilme süresi farklı olsun. Ödünç işlemini çok biçimli bir yöntemle gerçekleştirin.
- ◇ **Çöz:** Bir banka hesabı hiyerarşisi oluşturun: **Hesap** (taban), **ÇekingHesabı** ve **TasarrufHesabı** (türemiş). Her birinde para yatırma, çekme ve bakiye sorgulama işlemlerini uygulayın. Tasarruf hesabında minimum bakiye kısıtlamasını ekleyin.
- ◇ **Çöz:** **IYazılabilir** ve **IYazdırılabilir** adlı iki arayüz tanımlayın. Belge, Fatura ve Rapor sınıflarını bu arayüzleri farklı kombinasyonlarda uygulayarak oluşturun.
- ◇ **Çöz:** Bir ürün katalog sistemi tasarlayın: Ürün sınıfı kapsülleme kurallarına tam uygun olsun (**private** alanlar, **public** özellikler, doğrulama mantığı). Fiyat sıfır veya negatif girildiğinde istisna fırlatılsın.
- ◇ **Çöz:** Genişleme yöntemi (**extension method**) kullanarak **string** sınıfına 'Tersine çevir', 'Sözcük sayısını bul' ve 'İlk harfi büyüt' yöntemlerini ekleyiniz.
- ◇ **Düşün:** Bir televizyonu nesne olarak düşünürsek; televizyonun 'kanal numarası' bir durum (**state**) mudur yoksa bir davranış (**behavior**) mıdır? Peki 'kanal değiştirme' işlemi hangisidir?
- ◇ **Düşün:** Neden tüm kodlarımızı tek bir **namespace** içinde yazmıyoruz? İsim uzaylarının (**namespaces**) büyük ölçekli projelerdeki en büyük faydası nedir?
- ◇ **Düşün:** Bir sınıfın içinde hem **static** hem de **non-static** metotlar olabilir mi? Hangisi nesne üretilmeden çağrılabilir?